A Framework for Stochastic Differentiable Programming

Guillaume Verdon, Leo Tyrpak, Owen Lockwood, Seth Morton,
Alexander Neagoe, Anton Sugolov, Ian MacCormack, and Mirko Amico
Extropic Corporation, San Francisco, California 94111, USA
(Dated: August 14, 2026)

We introduce Parametrized Stochastic Circuits (PSCs), a gate-based intermediate representation for programmable stochastic dynamics in which typed local stochastic kernels with tunable parameters compose over explicit binary, categorical, and continuous wires, and `torx`, an open-source JAX framework for constructing, executing, and differentiating them. PSC's data types and stochastic kernels are chosen to align closely with the native operations exposed by emerging probabilistic hardware. In this way, stochastic algorithms can be designed directly in terms of the operations the hardware executes natively, so that the energy advantage arising at this level is not lost on mappings that introduce substantial decomposition, communication, or control overhead. We demonstrate the framework on a variety of example applications such as random walks on graphs, discrete diffusion, stochastic graph networks, jump diffusion and Ising sampling. We also report a hardware experiment in which probabilistic bits on the X0 subthreshold CMOS test chip, hosted by the XTR-0 desktop platform, provide physical randomness for Metropolis-Hastings and importance-sampling estimators, yielding estimates consistent with a software pseudorandom baseline.

arXiv:2608.01612v2 [cs.ET] 13 Aug 2026

I. INTRODUCTION

Many modern algorithms are built from parametrized stochastic maps. Diffusion models learn denoising transitions that turn noise into data [1], Monte Carlo methods chain transition kernels to estimate high-dimensional expectations [2], probabilistic circuits represent distributions through compositions of tractable factors [3, 4], and regime-switching models couple a latent discrete process to a continuous observable [5]. These methods share a compositional structure: local probabilistic operations are assembled into a global computation, and their parameters may be fixed or optimized against scalar objectives defined on samples or output distributions. A logical composition, however, does not by itself specify how its operations are realized. In conventional software this distinction is largely an implementation detail but on specialized stochastic hardware, it can determine the feasibility and cost of execution.

Emerging thermodynamic and probabilistic devices implement physical stochastic primitives at low energy cost. Some devices provide physical random sources that can be consumed by otherwise digital algorithms, such as probabilistic bits built from magnetic tunnel junctions [6, 7] or subthreshold CMOS circuits [8], whereas other devices implement families of stochastic kernels directly through their physical dynamics, such as Ising machines executing Gibbs sampling natively in silicon [9] and superconducting circuits realizing continuous-variable computations at thermodynamic equilibrium [10]. These devices differ in their supported variable types, transition families, connectivity, and control interfaces. Any advantage depends on the total cost of realizing the logical operations on the device. Lowering a high-level stochastic map through a complex sequence of physical primitives, communication steps, or digital computations may eliminate the potential advantage of the substrate. This motivates a hardware-near intermediate representation that specifies the logical composition of stochastic kernels while keeping the structure and cost of their physical realization visible.

Existing software frameworks for stochastic computing focus mainly on providing composable primitives at a high level of abstraction. Stochastic computation graphs formalize computations as directed graphs of deterministic operations and conditional random variables and give general rules for constructing unbiased gradient estimators [11]. Pyro [12] and NumPyro [13] provide composable modeling and inference primitives on differentiable array frameworks, while TensorFlow Probability exposes composable Markov-transition kernels for constructing and executing MCMC algorithms [14]. The frameworks establish that stochastic maps can be represented, composed and executed in multiple ways. However, their primitives are software constructs, distribution objects, tensor operations, and inference routines, with no direct correspondence to the typed variables and local stochastic kernels a device natively provides. Gate-and-circuit abstractions in quantum software frameworks such as TensorFlow-Quantum [15] and PennyLane [16] faced the same constraint, where the scarce resource in noisy quantum devices is coherence rather than energy, and settled on the circuit as the lowest level of abstraction that remains intelligible and differentiable while keeping the mapping to hardware simple. However, an analogous solution for stochastic computation has not yet appeared.

In this work, we introduce Parametrized Stochastic Circuits (PSCs), a gate-based model of programmable stochastic dynamics which serves as such an intermediate representation, together with `torx` [17], an open-source JAX [18, 19] framework for constructing, executing and differentiating them. A PSC is an ordered sequence of layers of stochastic gates acting on binary, categorical, and continuous wires, where each gate is a local conditional transition rule with tunable settings. Gates may be finite transition matrices, Markov kernels on contin-

uous spaces, or finite-time operators obtained by exponentiating local rate models. The primitives of a PSC, typed variables and local stochastic kernels with explicit supports are close to the components a device natively provides while leaving some freedom to lower the logical description given by a PSC to match exactly the hardware constraints. All stochastic effects are localized to the gates, so one can fix the program semantics by defining a logical PSC and then choose a variety of execution methods. Exact execution propagates the full output distribution when the state space allows it. Sampled execution provides trajectories drawn from the circuit and is compatible with software and hardware randomness providers, which can be exchanged without modifying the logical PSC. In the case of devices implementing kernels via their physical dynamics, one can compile the logical PSC to a hardware-compatible representation, as described in the `thermalizers` framework of the companion manuscript [20].

We demonstrate the framework on various example applications, from purely binary circuits to hybrid discrete-continuous ones, including random walks on graphs, Ising sampling, discrete diffusion, stochastic graph networks, and jump diffusion. We also report a hardware experiment in which calibrated probabilistic bits (pbits) on the X0 subthreshold CMOS test chip, hosted by the XTR-0 desktop platform, supply Bernoulli randomness to Metropolis-Hastings and importance-sampling estimators, with estimates consistent with a software pseudo-random baseline. The logical PSC is unchanged between the two runs, and the only difference between them is the randomness provider.

The remainder of the manuscript is organized as follows. Section II defines the computational primitives, stochastic kernels and their compositions. Section III defines PSCs, the elementary gate library, product-formula approximations, and the gradient estimation rules. Section IV describes the `torx` framework along with its software and hardware backends. Section V demonstrates the framework on the examples mentioned above, including the X0 hardware experiment, and the conclusions follow in Section VI. Appendices expand on details left out of the main text.

II. BACKGROUND AND PRELIMINARIES

This Section introduces the basic concepts which we build upon. We proceed at different levels of granularity, leading up to the gate-based circuit model of PSCs of Section III. In Section II A we define the fundamental units of stochastic computation that we work with. Section II B defines stochastic kernels as the basic operations on data, the maps that transform states. We recall the definition of a Markov chain, a sequence of stochastic kernels, in Section II C. Finally, composition of stochastic kernels into Directed Acyclic Graphs (DAGs) is covered in Section II D.

A. Primitives and notation

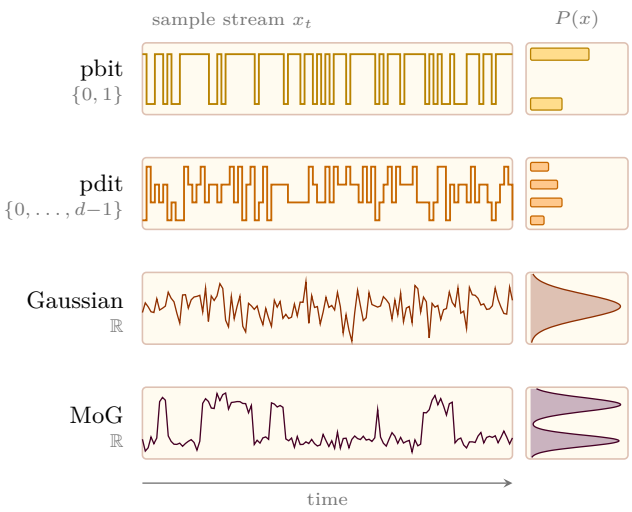


FIG. 1: The data primitives of Table I. Each row shows an example of a sample stream x_t together with the distribution $P(x)$ from which the samples are drawn. A pbit carries a distribution over two states, a pdit a distribution over d states, and a pmode a probability density over $\mathbb{R}$; the Gaussian and mixture-of-Gaussians rows are two examples of distributions carried by a pmode wire.

We consider three data types, distinguished by their state space, summarized in Table I and illustrated in Figure 1. Throughout, we refer to a variable carrying one of these data types as a *wire*, in analogy with circuit diagrams; the circuits acting on wires are defined in Section III.

TABLE I: Data primitives and their state spaces.

Data type	State space $\mathcal{X}$
pbit (2 states)	$\{0,1\}$
pdit (d states)	$\{0,\ldots,d-1\}$
pmode (N dimensions)	$\mathbb{R}^N$

A *pbit* is the binary case: its state space is $\mathcal{X} = \{0,1\}$ and its state is a length-2 probability vector $\rho = (\rho_0,\rho_1) \in \Delta_1$, where $\Delta_{d-1} = \{\rho \in \mathbb{R}^d : \rho_a \geq 0,\ \sum_a \rho_a = 1\}$ denotes the probability simplex on d states.

A *pdit* generalizes this to d states [21]: its state space is $\mathcal{X} = \{0,\ldots,d-1\}$ and its state is a length-d probability vector $\rho \in \Delta_{d-1}$.

A *pmode* is a continuous wire with state space $\mathcal{X} = \mathbb{R}^N$; its state is a probability density $\rho(x) \geq 0$ satisfying $\int_{\mathbb{R}^N} \rho(x)\,dx = 1$. The scalar case $N = 1$ is assumed when N is left unspecified. While a pmode can carry an arbitrary continuous distribution, the elementary pmode gates of Section III A act on Gaussian

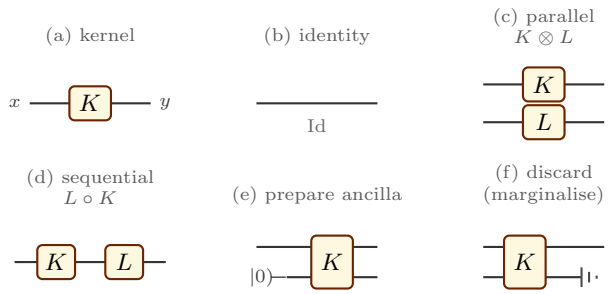


FIG. 2: Diagrammatic notation for stochastic kernels, matching the panel labels above. **(e)** prepares an ancilla in the fixed state $|0)$; **(f)** discards (marginalizes) a wire, denoted by a ground symbol.

densities for tractability; non-Gaussian generator-derived gates (e.g. PGBM, PCIR) are catalogued in Appendix C.

Combining two state spaces $\mathcal{X}_1$ and $\mathcal{X}_2$ gives a joint state space $\mathcal{X}_1 \times \mathcal{X}_2$ whose elements are pairs (a_1, a_2). A register of n pbits, for example, has state space $\{0,1\}^n$ and state $\rho \in \Delta_{2^n-1}$.

We adopt Dirac-style notation for deterministic states: $|a_1 \dots a_n)$ denotes the distribution on $\{0,1\}^n$ that assigns probability 1 to the string $(a_1, \dots, a_n)$,

$$|a_1 \dots a_n) \;=\; \delta_{(\,\cdot\,),(a_1,\dots,a_n)}. \tag{1}$$

Every distribution on $\{0,1\}^n$ is then a convex combination $\rho = \sum_{a_1,\dots,a_n} \lambda_{a_1\dots a_n} |a_1 \dots a_n)$. This notation is inspired by quantum computation for convenience and used only where it aids clarity.

With the states fixed, the next step is to define the maps that act on them.

B. Stochastic Kernels

A *stochastic kernel* with input space $\mathcal{X}$ and output space $\mathcal{Y}$ is a function $K : \mathcal{Y} \times \mathcal{X} \to \mathbb{R}$ where $K(y \mid x)$ is the conditional density of outputting y given input x. We write $y \sim K(\cdot \mid x)$ for a draw from this conditional. It must satisfy positivity $K(y \mid x) \geq 0$ and normalization for each fixed x,

$$\int_{\mathcal{Y}} K(y \mid x)\, dy \;=\; 1, \tag{2}$$

where the integral is a finite sum when $\mathcal{Y}$ is discrete. In the discrete case, (2) is equivalent to *column-stochasticity*: each column of the matrix $\big(K(y \mid x)\big)_{y,x}$ sums to 1. A stochastic kernel acts on distributions by

$$\rho_{\text{out}}(y) = (K\,\rho_{\text{in}})(y) = \int_{\mathcal{X}} K(y \mid x)\, \rho_{\text{in}}(x)\, dx, \tag{3}$$

which for discrete states is the matrix vector product $\rho_{\text{out}}(y) = \sum_x K(y \mid x)\, \rho_{\text{in}}(x)$.

For any bounded measurable function $f : \mathcal{X} \to \mathbb{R}$, the expectation with respect to ρ is written uniformly as

$$\langle f, \rho \rangle \;=\; \begin{cases} \displaystyle\sum_{a \in \mathcal{X}} f(a)\, \rho_a \;=\; f^\top \rho & \text{(discrete wire)}, \\ \displaystyle\int_{\mathbb{R}} f(x)\, \rho(x)\, dx & \text{(pmode wire)}. \end{cases} \tag{4}$$

For state spaces that combine discrete and continuous factors, the joint expectation expands to a sum-integral:

$$\langle f, \rho \rangle = \sum_{a \in \mathcal{X}_{\text{d}}} \int_{\mathcal{X}_{\text{c}}} f(a,x)\, \rho(a,x)\, dx, \tag{5}$$

where $\mathcal{X}_{\text{d}}$ and $\mathcal{X}_{\text{c}}$ are the discrete and continuous parts of the state space, respectively. Applying K to ρ transforms expectations as $\langle f, \rho \rangle \mapsto \langle f, K\rho \rangle = \langle K^\top f, \rho \rangle$, so a kernel acts forward on states and backward on observables. This dual action is what allows the circuit objectives of Section III C to be differentiated one gate at a time.

We build stochastic programs by combining single kernels as our building blocks. Figure 2 fixes the diagrammatic notation used throughout. A kernel is a box on a wire, read from input x on the left to output y on the right (Figure 2(a)), with the identity kernel $\mathbb{I}(x' \mid x) = \delta_{x',x}$, which leaves the state unchanged, drawn simply as an empty wire (Figure 2(b)). We can then compose kernels either in series and in parallel.

Parallel composition (Figure 2(c)) places kernels on disjoint wires. For $K : \mathcal{Y} \times \mathcal{X} \to \mathbb{R}$ and $L : \mathcal{Y}' \times \mathcal{X}' \to \mathbb{R}$ acting on disjoint state spaces,

$$(K \otimes L)(y, y' \mid x, x') = K(y \mid x)\; L(y' \mid x'). \tag{6}$$

Since each factor integrates to one, so does their product, and parallel composition sends probability distributions to probability distributions.

Sequential composition (Figure 2(d)) places kernels one after the other. For $K : \mathcal{Z} \times \mathcal{X} \to \mathbb{R}$ followed by $L : \mathcal{Y} \times \mathcal{Z} \to \mathbb{R}$,

$$(L \circ K)(y \mid x) = \int_{\mathcal{Z}} L(y \mid z)\, K(z \mid x)\, dz, \tag{7}$$

which in the discrete case is ordinary matrix multiplication. Since each kernel integrates to one over its output, composition sends probability distributions to probability distributions, with $\rho_{\text{out}} = (L \circ K)\, \rho_{\text{in}} = L(K\, \rho_{\text{in}})$. In particular, the kernels with the same input and output space $\mathcal{X}$ form a semigroup under composition with the identity $I(x' \mid x) = \delta_{x'=x}$.

Finally, two operations can change the numbers of wires. Figure 2(e) prepares an *ancilla*, a fresh wire initialized to a fixed state $|0)$ and fed into a gate, and Figure 2(f) *discards* a wire by marginalizing over its values. These four operations, $\circ$, $\otimes$, preparation, and discarding, suffice to build every kernel considered in this paper.

C. Discrete and continuous-time Markov chains

By iterating the compositions introduced in Section II B one can generate dynamical behavior. Here we briefly review the resulting discrete-time Markov chains and their continuous-time limit, for a more complete treatment see [22].

The repeated application of a fixed kernel defines a *discrete-time Markov chain*. Starting from ρ_0, the state after n steps is

$$\rho_n = K^n \rho_0, \qquad K^n = \underbrace{K \circ \cdots \circ K}_{n}. \tag{8}$$

At the level of sample paths, $(X_n)_{n\geq 0}$ is a stochastic process whose one-step transition satisfies $P(X_{n+1} = x_{n+1} \mid X_0 = x_0, \ldots, X_n = x_n) = K(x_{n+1} \mid x_n)$, so the future depends on the past only through the current state. A distribution π is *stationary* for K if $K\pi = \pi$. Under standard ergodicity conditions (irreducibility and aperiodicity in the discrete-state case), $\rho_n \to \pi$ as $n \to \infty$ regardless of ρ_0. We call K the transition kernel of this Markov chain. We note that in standard literature the transition kernel $P = K^\top$ is right multiplied to the probability vector giving ρP unlike here where we left multiply.

Continuous-time dynamics arise from the same construction when the step becomes infinitesimal. Consider a kernel that deviates from the identity by a small step τ,

$$K_\tau = \mathbb{I} + \tau\, Q, \tag{9}$$

where column-stochasticity of K_τ requires Q to have off-diagonal entries $Q_{ij} \geq 0$, the transition rate from state j to state i, and diagonal entries $Q_{ii} = -\sum_{j\neq i} Q_{ij}$, so that each column sums to zero. We call Q a *rate matrix* (or *generator*). Composing N such steps over a time t and letting $N \to \infty$ gives

$$\rho(t) = \lim_{N\to\infty} \big(\mathbb{I} + Q\, t/N\big)^N \rho_0 = e^{Qt}\, \rho_0, \tag{10}$$

which is the solution of the *forward Kolmogorov equation* $\mathrm{d}\rho/\mathrm{d}t = Q\,\rho$. A stationary distribution satisfies $Q\pi = 0$. The family $K_t = e^{Qt}$ is column-stochastic for every $t \geq 0$ and forms a one-parameter semigroup within the semigroup of kernels of Section II B. Fixing a step size t recovers the discrete-time picture. Both the Euler kernel $\mathbb{I} + \tau Q$ and the exact exponential e^{Qt} reappear as gates. The former in the product formulas of Section III B, the latter in generator-derived gates such as PIsing in Section III A.

To express larger parametrized probabilistic programs, we next allow many kernels to be organized in an acyclic graph and composed through their shared variables.

D. Directed Factor Graphs

The compositions of Section II B build kernels in series and in parallel, but many probabilistic models are specified more loosely, as a network of conditional distributions in which several kernels consume and produce shared variables. A Directed Factor Graph (DFG) captures this general case: it is a directed acyclic graph of stochastic kernels, where the output of each kernel is fed as input to one or more subsequent kernels. This structure is an instance of a directed graphical model [23], where nodes represent stochastic kernels, directed edges carry the variables passed between them, and internal variables are marginalized out to yield an overall conditional distribution from the designated inputs to the designated outputs. We use the DFG as the internal representation underlying the PSCs of Section III, and to derive the gradient estimators of Section III C.

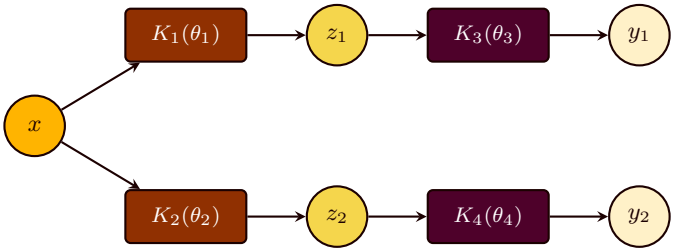


FIG. 3: A directed factor graph of stochastic kernels. Variable nodes (circles) are connected via factor nodes (rectangles) representing the kernels $K_i(\theta_i)$.

Each kernel $K_i(\theta_i) : \mathcal{Y} \times \mathcal{X} \to \mathbb{R}$ is indexed by parameters $\theta_i \in \Theta_i \subseteq \mathbb{R}^{d_i}$, with entries $K_i(y \mid x; \theta_i)$. We shall sometimes omit the dependence on θ_i when convenient. We call K_i a *parent* of K_j if there is a directed edge from K_i to K_j, i.e. an output of K_i serves as an input to K_j. We assume a hierarchical (also called topological) ordering: all parents appear before their children, so if K_i is a parent of K_j, then $i < j$. This ordering means the DAG can always be evaluated as the sequential composition

$$\mathcal{K}(\theta) = K_n(\theta_n) \circ \cdots \circ K_1(\theta_1), \tag{11}$$

parametrized by $\theta = (\theta_1, \ldots, \theta_n) \in \Theta$ and the state evolves through the graph as

$$\begin{aligned} \rho_j &= K_j\, \rho_{j-1}, \\ \rho_{\text{out}} &= \mathcal{K}\, \rho_{\text{in}} = K_n \cdots K_1\, \rho_{\text{in}}. \end{aligned} \tag{12}$$

The corresponding sample path is $z^{(0)}, \ldots, z^{(n)}$ with

$$\begin{aligned} z^{(0)} &\sim \rho_{\text{in}}, \\ z^{(j)} &\sim K_j(\,\cdot \mid z^{(j-1)}), \qquad j = 1, \ldots, n, \end{aligned}$$

so that $z^{(n)} \sim \rho_{\text{out}}$. We call $z^{(0)}$ the input, $z^{(n)}$ the output, and $z^{(1)}, \ldots, z^{(n-1)}$ the latent variables. We note that, as shown in Figure 3, the same DAG may admit several hierarchical orderings. We do not assume access to a closed-form expression for the conditional distribution, we assume only that each kernel K_j can be sampled efficiently. A smooth parameterization is typically chosen in learning settings so that the gradients $\nabla_{\theta_i} K_i(y \mid x)$ can be computed.

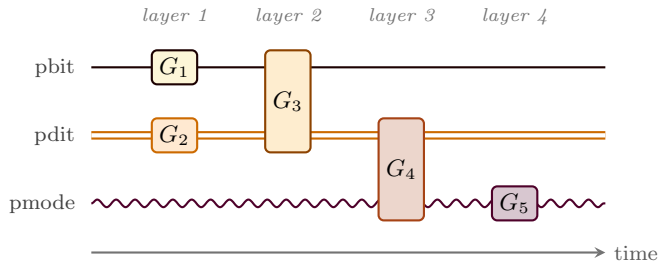


FIG. 4: Within layer 1, gates G_1 and G_2 execute in parallel. Layer 2 contains a cross-type gate G_3 coupling the pbit and the pdit. Layer 3 contains a cross-type gate G_4 coupling the pdit and the pmode. Layer 4 acts on the pmode.

A DFG places no constraint on how kernels are scheduled beyond the topological ordering. The PSC of Section III equips this structure with a fixed register of typed wires and ordered layers of gates with disjoint supports. The layered structure makes parallel execution explicit, thus making compilation to hardware more obvious. Conversely, every PSC unrolls to a DFG by treating each gate as a kernel and each wire segment as a shared variable.

III. PARAMETRIZED STOCHASTIC CIRCUITS

A Parametrized Stochastic Circuit is the circuit-shaped specialization of the directed factor graph in Section II D. Its kernels are grouped into ordered layers: kernels in the same layer have disjoint state spaces and execute in parallel, while layers compose in series to define a global stochastic map on the full state space. Within a PSC, wires carry states of the data types described in Sec. II A. All stochasticity is localized to the gate kernels, while the wires themselves only transmit the variables' state between layers. PSCs give a hardware-near model for stochastic computation. A logical circuit defined within this model can be compiled to backends with different execution rules, such as full state-space simulation and sample-based execution with software-sourced pseudo-randomness and hardware-sourced randomness (via the XTR-0 platform, Sec. IV B). The companion manuscript [20] also explores execution of PSCs on thermodynamic hardware (the Z1 chip of Fig. 7) by compiling them to an energy-based model intermediate representation.

We now give more precise definitions of the components of a PSC. A *register* is the full collection of wires acted on by the circuit. Using the notation of Section II A, let $\mathcal{W}$ be the set of wires in the register, $\mathcal{X}_w$ the state space of wire w, and $\mathcal{X}_I = \prod_{w\in I} \mathcal{X}_w$ the product state space of a subset $I \subseteq \mathcal{W}$. A local stochastic gate supported on I is a stochastic kernel

$$G_I(\theta) : \mathcal{X}_I \times \mathcal{X}_I \to \mathbb{R}, \tag{13}$$

with tunable parameters θ. When an operation changes the number of wires, it is represented in the fixed-register picture by preparing ancilla wires and discarding unused wires, using the primitives of Section II B. We define a set of *elementary* gates in Section III A. These are the primitives from which more complex kernels are built by composition in series and parallel.

A *layer* is a collection of gates with pairwise disjoint supports, as illustrated in Figure 4, where gates G_1 and G_2 occupy the same layer and execute in parallel. If $J = \{I_1, \ldots, I_m\}$ is the collection of active supports in the layer, then

$$L_J(\theta) = \bigotimes_{r=1}^{m} G_{I_r}(\theta_r). \tag{14}$$

Wires outside the active support pass unchanged to the next layer:

$$\widetilde{L}_J(\theta) = L_J(\theta) \otimes \mathbb{I}_{(\bigcup_{r=1}^m I_r)^c}. \tag{15}$$

This embedding, together with any necessary wire permutation, is understood implicitly. A *parametrized stochastic circuit*

$$\mathcal{G}(\theta) = \widetilde{L}_{J_k}(\theta_k) \cdots \widetilde{L}_{J_1}(\theta_1) : \mathcal{X}_\mathcal{W} \times \mathcal{X}_\mathcal{W} \to \mathbb{R}, \tag{16}$$

is a stochastic kernel on the register $\mathcal{X}_\mathcal{W}$: the register's state is a distribution ρ over $\mathcal{X}_\mathcal{W}$, while any single realization (sample) of the register takes a definite, deterministic value in $\mathcal{X}_\mathcal{W}$. It is composed of layers $\widetilde{L}_{J_j}(\theta_j)$, and the composition is read from right to left: $\widetilde{L}_{J_1}(\theta_1)$ acts first, then $\widetilde{L}_{J_2}(\theta_2)$, and so on until $\widetilde{L}_{J_k}(\theta_k)$ acts last.

A. Elementary Gates

Gates of a PSC are defined here through three possible constructions. The first is the direct specification of a stochastic matrix. The second is via convex combinations of deterministic operations. For any deterministic gate B, the mixture $(1-p)\,\mathbb{I} + p\,B$ is implemented by preparing an auxiliary pbit in the state $\rho_{\text{aux}} = (1-p)|0) + p|1)$, applying B conditioned on the auxiliary state $|1)$ and nothing conditioned on $|0)$, and discarding the auxiliary pbit. A convenient parametrization for learning passes the mixture weight through a sigmoid,

$$G(\theta) = \big(1 - \sigma(\theta)\big)\,\mathbb{I} + \sigma(\theta)\,B, \qquad \sigma(\theta) = \frac{1}{1+e^{-\theta}}, \tag{17}$$

so that $p = \sigma(\theta) \in (0,1)$ for every $\theta \in \mathbb{R}$. This is the gate family to which the parameter-shift rule of Section III C applies. The third construction method uses exponentials of the local generators of Section II C. Continuous-time finite-state Markov models are generated by rate matrices whose exponentials are stochastic transition maps [22]. To define a gate, choose a finite-state rate matrix Q and exponentiate it to obtain a generator-derived

stochastic kernel $K(t) = \exp(tQ)$. Markov-type Lie theory provides a reference point for such generator-based operator families [24]. In the generator-derived subfamily, PSCs recover the operator-splitting picture where local Markov generators are exponentiated to finite-time stochastic kernels and then composed in time.

Let us now define some elementary gates for stochastic circuits, organized by wire type (further gates are cataloged in the Appendices A and C).

a. pbit and pdit The probabilistic NOT (PNOT) flips the state of a pbit with probability $p \in [0, 1]$:

$$\mathsf{PNOT}(p) = \begin{pmatrix} 1-p & p \\ p & 1-p \end{pmatrix}. \tag{18}$$

At $p = 0$ this is the identity, at $p = 1$ it is the deterministic NOT, and at $p = \frac{1}{2}$ it produces a uniform distribution regardless of input. Applying $\mathsf{PNOT}(p)$ to a single-pbit with distribution $\rho = (\alpha,\, \beta)^\top$ yields

$$\mathsf{PNOT}(p)\,\rho = \begin{pmatrix} (1-p)\,\alpha + p\,\beta \\ p\,\alpha + (1-p)\,\beta \end{pmatrix}, \tag{19}$$

exchanging probability mass between $|0)$ and $|1)$.

Gates can also act on multiple binary wires, the probabilistic swap (PSWAP) is one such example. It operates on two pbits, exchanging their states with probability $p \in [0, 1]$:

$$\mathsf{PSWAP}(p)\,|ab) = (1-p)\,|ab) + p\,|ba). \tag{20}$$

Like PNOT, PSWAP is a convex combination of the identity and a deterministic operation.

As an example of a generator-derived gate we take PIsing, which defines a local two-spin Glauber kernel [25]. Given parameters $\theta = (J, h_1, h_2, \beta, \Delta t)$, the energy of a two-pbit configuration $\mathbf{s} = (s_1, s_2) \in \{0, 1\}^2$ is

$$E(\mathbf{s}) = -J(2s_1 - 1)(2s_2 - 1) - h_1(2s_1 - 1) - h_2(2s_2 - 1). \tag{21}$$

The associated Glauber dynamics has rate matrix Q with off-diagonal entries

$$Q_{ab} = \sigma\big[-\beta\big(E(\mathbf{s}_a) - E(\mathbf{s}_b)\big)\big] \tag{22}$$

for single-spin-flip transitions $a \neq b$, zero otherwise, and diagonal $Q_{bb} = -\sum_{a \neq b} Q_{ab}$. The gate is

$$\mathsf{PIsing}(\theta) = \exp(\Delta t\, Q), \tag{23}$$

which is column-stochastic for $\Delta t \geq 0$. We expand upon this in our companion paper [20] which focuses exclusively on energy-based transition kernels.

b. pmode A pmode gate is a kernel $K(y \mid x)$ on $\mathbb{R}^N$. The elementary directly specified gate is the *affine Gaussian gate* $(M, \boldsymbol{d}, \Delta)$,

$$X \mapsto MX + \boldsymbol{d} + \varepsilon, \qquad X \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma), \quad \varepsilon \sim \mathcal{N}(0, \Delta), \tag{24}$$

which maps $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$ to $\mathcal{N}(M\boldsymbol{\mu} + \boldsymbol{d},\, M\Sigma M^\top + \Delta)$, and special cases such as displacements, scalings, and rotations are cataloged in Appendix C. The generator-derived example is the Ornstein–Uhlenbeck gate $\mathsf{POU}(\gamma, D, t)$, obtained by exponentiating the Fokker–Planck generator of the SDE $\mathrm{d}X_t = -\gamma X_t\,\mathrm{d}t + \sqrt{2D}\,\mathrm{d}W_t$; it combines deterministic contraction with noise injection and plays for pmodes the role PIsing plays for pbits (Appendix C). These Gaussian primitives support a broad family of continuous-variable algorithms, ranging from Gaussian sampling to deep neural networks, whose execution on equilibrium-based superconducting thermodynamic hardware is demonstrated in Ref. [10], and could be readily implemented in CMOS using X0 or in Z1 via `thermalizers` compilation [20].

c. Hybrid pdit/pmode gates Many stochastic systems couple a discrete state to a continuous one. PSCs accommodate this through gates acting on the product space $\{0, \ldots, d-1\} \times \mathbb{R}^N$ with the compositional rules of Section II B unchanged.

Controlled shift $\mathsf{CShift}(\boldsymbol{\alpha})$: a pdit conditions a deterministic displacement on the pmode,

$$\mathsf{CShift}(\boldsymbol{\alpha}) : \ |k) \otimes p(x) \ \longmapsto \ |k) \otimes p(x - \alpha_k), \tag{25}$$

where $\boldsymbol{\alpha} = (\alpha_0, \ldots, \alpha_{d-1})$ gives the shift selected by each pdit state. The pmode-side action when the pdit is held in state k is a $\mathsf{PDisp}(\alpha_k)$, the displacement gate defined in Appendix C.

Controlled scale $\mathsf{CScale}(\boldsymbol{r})$: the multiplicative analog,

$$\mathsf{CScale}(\boldsymbol{r}) : \ |k) \otimes p(x) \ \longmapsto \ |k) \otimes e^{-r_k} p(e^{-r_k} x), \tag{26}$$

rescales the pmode by e^{r_k} when the pdit is in state k.

Mixture-of-Gaussians $\mathsf{MoG}(\boldsymbol{\pi}, \boldsymbol{\alpha})$: preparing a pdit ancilla with probability distribution $\boldsymbol{\pi} = (\pi_0, \ldots, \pi_{d-1}) \in \Delta_{d-1}$, applying $\mathsf{CShift}(\boldsymbol{\alpha})$ to a Gaussian pmode $X \sim \mathcal{N}(\mu, \sigma^2)$, and marginalizing the ancilla yields the Gaussian mixture

$$p_{X'}(x) = \sum_{k=0}^{d-1} \pi_k\, \mathcal{N}(x; \mu + \alpha_k, \sigma^2), \tag{27}$$

a universal density approximator under standard regularity conditions. Combining CScale with CShift yields the heteroscedastic variant $\sum_k \pi_k\, \mathcal{N}(e^{r_k}\mu + \alpha_k,\, e^{2r_k}\sigma^2)$. Note that MoG is precisely the construction of Figure 2(e)–(f): a prepared ancilla, a controlled gate, and a discard.

B. Trotterization

In Section III A we have introduced gates that act locally on a few wires at a time. However, in most cases one is interested in the dynamics generated by a sum of local terms

$$Q = \sum_{j=1}^{k} Q_j, \tag{28}$$

where each Q_j is supported on a few wires while the exponential e^{Qt} is a kernel on the full register, in general not equal to any product of local gates.

Trotterization [26] allows us to approximate e^{Qt} by a circuit of local gates applied over N steps of size $\tau = t/N$, each implementing either the exact local exponential $e^{Q_j\tau}$ or the Euler kernel $\mathbb{I} + \tau Q_j$ of Eq. (9). For example, in Section V A the graph Laplacian decomposes into one generator per edge, and each factor is a single PSWAP gate.

Given access to exact local exponentials, one can approximate e^{Qt} via *Lie–Trotter splitting* [26], which splits the exponential of the sum into a product of local exponentials,

$$e^{Qt} = \left(e^{Qt/N}\right)^N \approx \left(\prod_{j=1}^{k} e^{Q_j t/N}\right)^N . \tag{29}$$

The error of a single step is controlled by the commutators $[Q_i, Q_j]$, and vanishes when all generators commute. The single-step error is of order $O(t^2k^2/N^2)$, and since the errors of the N steps accumulate at most additively (the telescoping bound), the global error is $O(t^2k^2/N)$. This is a first-order method, doubling N halves the global error. The splitting error can be reduced by symmetrizing the product. The second-order *Strang splitting* [27] approximates

$$\begin{aligned} e^{Qt/N} \approx & e^{Q_1 t/2N} e^{Q_2 t/2N} \cdots e^{Q_{k-1} t/2N} e^{Q_k t/N} \\ & e^{Q_{k-1} t/2N} \cdots e^{Q_2 t/2N} e^{Q_1 t/2N}, \end{aligned} \tag{30}$$

which cancels the leading commutator error by symmetry and achieves a local error of $O(t^3k^3/N^3)$, hence a global error of $O(t^3k^3/N^2)$, at the cost of $2k-1$ rather than k gate applications per step. Higher-order product formulas [28] exist but are not needed here.

When the exact local exponentials are not available as gates, each factor can be replaced by its Euler kernel,

$$e^{Qt} = \left(e^{Qt/N}\right)^N \approx \left(\prod_{j=1}^{k} (\mathbb{I} + Q_j\, t/N)\right)^N , \tag{31}$$

and the single-step error decomposes into an Euler part and a splitting part,

$$\begin{aligned} \left\| e^{Qt/N} - \prod_{j=1}^{k} (\mathbb{I} + Q_j\, t/N) \right\| \le & \underbrace{\left\| e^{Qt/N} - (\mathbb{I} + Qt/N) \right\|}_{O(t^2/N^2)} \\ + & \underbrace{\left\| (\mathbb{I} + Qt/N) - \prod_{j=1}^{k} (\mathbb{I} + Q_j\, t/N) \right\|}_{O(t^2k^2/N^2)}, \end{aligned}$$

giving the same $O(t^2k^2/N)$ global error as Lie–Trotter. These bounds hold for any submultiplicative matrix norm; the implicit constant is proportional to the maximum pairwise commutator norm $\max_{i\neq j} \|[Q_i, Q_j]\|$.

The number of factors k entering these bounds can often be reduced by grouping. If some of the generators commute pairwise, one constructs the non-commutation graph on $\{Q_j\}$ and colors it with χ colors, so that generators of the same color commute. Within a color class the product is exact, $e^{G_c\tau} = \prod_{j\in\mathcal{C}_c} e^{Q_j\tau}$ with $G_c = \sum_{j\in\mathcal{C}_c} Q_j$, so Trotterizing the χ grouped generators replaces k by χ in all the bounds above. For generators tiled over the edges of a graph this is an edge coloring, the case used in Section V A.

In practice the choice of formula reduces to what is available as a gate. When only Euler kernels $\mathbb{I} + Q_j\tau$ are available, both splittings incur an additional $O(kt^2/N)$ Euler error that dominates at any practical N, so Lie–Trotter is simpler and sufficient. When exact local exponentials are available, Strang is preferable for $N \gtrsim kt$ steps, where its error $O(k^3t^3/N^2)$ drops below the Lie–Trotter error $O(k^2t^2/N)$.

C. Differentiation & Optimization

Here we focus on methods for training a PSC by gradient descent on a scalar objective estimated from samples, which requires gradient estimators that are themselves computable by running circuits. Figure 5 illustrates this hybrid classical-stochastic loop. We present three such estimators, in increasing order of the structure they assume on the gates: the REINFORCE estimator, valid for any smoothly parametrized gate; the parameter-shift rule, valid for the sigmoid-mixture gates of Eq. (17); and an energy-based estimator, valid for gates sampling from a Boltzmann conditional. Following Section II D, we derive them at the generality of the underlying DFG. We write the circuit as gates $G_j \equiv G_{I_j}(\theta_j)$, $j = 1, \ldots, k$, applied in topological order; running the circuit once produces a latent trajectory $(z^{(0)}, \ldots, z^{(k)})$ with $z^{(0)} \sim \rho_{\text{in}}$ and $z^{(j)} \sim G_j(\,\cdot \mid z^{(j-1)})$, so that $z^{(k)} \sim \rho_{\text{out}}(\theta)$.

The objective is the expectation of a readout function f under the output distribution $\rho_{\text{out}}(\theta) = G_k \cdots G_1\, \rho_{\text{in}}$,

$$\begin{aligned} J(\theta) = \mathbb{E}_{Y\sim\rho_{\text{out}}(\theta)}[f(Y)] &= \langle f, \rho_{\text{out}}(\theta)\rangle \\ &= \langle f,\ G_k \cdots G_1\, \rho_{\text{in}}\rangle, \end{aligned} \tag{32}$$

which we want to optimize with respect to θ. Since every G_j is linear in the state, the derivative follows from the product rule,

$$\partial_{\theta_j} J = \langle f,\ G_k \cdots G_{j+1}\, (\partial_{\theta_j} G_j)\, G_{j-1} \cdots G_1\, \rho_{\text{in}}\rangle, \tag{33}$$

where the dual action of Section II B moves the fixed gates onto the observable side. Writing the derivative of a kernel entrywise as $(\partial_{\theta_j} G_j)(y \mid x) = G_j(y \mid x)\, \partial_{\theta_j} \log G_j(y \mid x)$ turns Eq. (33) into an expectation over trajectories,

$$\partial_{\theta_j} J = \mathbb{E}_{(z^{(0)},\ldots,z^{(k)})}\left[f(z^{(k)})\, \partial_{\theta_j} \log G_j(z^{(j)} \mid z^{(j-1)}) \right], \tag{34}$$

the REINFORCE (or score-function) estimator [29]: drawing S independent trajectories and averaging the bracket gives an unbiased estimate of the full gradient from a single batch of circuit runs. The entrywise identity requires $G_j(z^{(j)} \mid z^{(j-1)}) > 0$ on the support of the trajectory distribution, which holds for the gate library of Section III A provided parameter values are kept away from the boundary $p \in \{0, 1\}$. In the case of the sigmoid parametrization of Eq. (17), this is enforced automatically.

a. Parameter shift rule estimator Consider the sigmoid-mixture gates $G(\theta) = (1 - \sigma(\theta))\,\mathbb{I} + \sigma(\theta)\,B$ of Eq. (17). Taking derivatives,

$$\frac{\partial G(\theta)}{\partial \theta} = \sigma'(\theta)\,(B - \mathbb{I}). \tag{35}$$

Since $\sigma'(\theta) = \sigma(\theta)\big(1 - \sigma(\theta)\big)$, the derivative is itself a difference of two gates of the same family. Defining the shifted parameter θ^- by $\sigma(\theta^-) = \sigma(\theta)^2$, that is

$$\theta^- = -\ln\big((1 + e^{-\theta})^2 - 1\big), \tag{36}$$

an explicit computation gives the logarithmic shift identity

$$\frac{\partial G(\theta)}{\partial \theta} = G(\theta) - G(\theta^-), \tag{37}$$

which expresses $\partial_\theta G$ as a difference of two executable gates.

If gate G_j is of the specified form, using (33) we can express the gradient as a difference of two circuit expectations,

$$\left.\frac{\partial \langle f, \rho_{\text{out}}(\theta)\rangle}{\partial \theta_j}\right|_{\theta_j = \theta_j^*} = \mathbb{E}_{X \sim \rho(\theta^*)}[f(X)] - \mathbb{E}_{\widetilde{X} \sim \rho(\theta^{*-})}[f(\widetilde{X})], \tag{38}$$

where $\rho(\theta^*)$ is the output distributions induced by running the original circuit and $\rho(\theta^{*-})$ the shifted circuit with parameter θ_j changed to,

$$\theta_j^{*-} = -\ln\big((1 + e^{-\theta_j^*})^2 - 1\big). \tag{39}$$

Drawing S iid samples $X_i \sim \rho_{\text{out}}(\theta^*)$ and $\widetilde{X}_i \sim \rho_{\text{out}}(\theta^{*-})$ gives the unbiased estimators

$$\widehat{g}_j = \frac{1}{S}\sum_{i=1}^{S} f(X_i) - \frac{1}{S}\sum_{i=1}^{S} f(\widetilde{X}_i), \tag{40}$$

with variance $\mathrm{Var}(\widehat{g}_j) = (\mathrm{Var}_{p_j^+} f + \mathrm{Var}_{p_j^-} f)/S$. The rule has two limitations. First, it applies only to gates of the sigmoid-mixture form of Eq. (17). Second, each parameter requires a separate shifted circuit evaluation, so unlike REINFORCE the full gradient cannot be obtained from a single batch of runs. However, only the gates downstream of G_j in the DFG need to be resampled; the remaining samples of the latent trajectory can be reused.

b. Energy-Based Kernels Suppose one of the kernels $G(\theta)$ samples from a Boltzmann distribution, i.e., the conditional distribution $G(\theta)(y \mid x)$ is obtained by conditioning a joint distribution with density

$$p_\theta(x, y) = \frac{1}{Z(\theta)} e^{-E(x,y;\theta)}. \tag{41}$$

For a fixed input x, the resulting kernel is

$$G(\theta)(y \mid x) = \frac{e^{-E(x,y;\theta)}}{Z(x;\theta)}, \qquad Z(x;\theta) = \int_{\mathcal{Y}} e^{-E(x,y';\theta)}\,\mathrm{d}y', \tag{42}$$

where the integral is replaced by a sum when $\mathcal{Y}$ is discrete.

We can then compute the gradient of the log-conditional explicitly:

$$\nabla_\theta \log G(\theta)(y \mid x) = -\nabla_\theta E(x, y; \theta) + \mathbb{E}_{y' \sim G(\theta)(\cdot \mid x)}\left[\nabla_\theta E(x, y'; \theta)\right]. \tag{43}$$

The expectation in the second term is over the output y' with the input x held fixed.

If gate G_j is of the specified form, using Eq. (34) gives

$$\nabla_{\theta_j} J(\theta) = \mathbb{E}_{(z^{(0)},\ldots,z^{(k)})}\left[f\big(z^{(k)}\big)\left(-\nabla_{\theta_j} E\big(z^{(j-1)}, z^{(j)}; \theta_j\big) + \mathop{\mathbb{E}}_{z' \sim G_j(\cdot \mid z^{(j-1)})}\left[\nabla_{\theta_j} E\big(z^{(j-1)}, z'; \theta_j\big)\right]\right)\right] \tag{44}$$

Thus, to estimate the gradient, one runs the full circuit to obtain the latent trajectory $(z^{(0)}, \ldots, z^{(k)})$ and draws an independent auxiliary sample

$$z^{(j)\prime} \sim G_j(\theta_j)(\cdot \mid z^{(j-1)})$$

to estimate the second term. The trajectory sample $z^{(j)}$ can be reused for the first term, so only gate G_j, clamped at the same input $z^{(j-1)}$, must be run again. This estimator is inexpensive but, as a score-function estimator, can suffer from high variance [30].

IV. TORX FRAMEWORK

A. Execution Interface

`torx` is a JAX-based library [18, 19] for constructing, executing, and training PSCs. Circuits are defined by specifying a certain combination of kernels. A circuit is then lowered to its underlying directed factor graph, a DAG of kernels each exposing a *sample* method. Given input values for a kernel, *sample* returns output values drawn from the kernel's conditional distribution. Training is implemented following the hybrid loop of Figure 5.

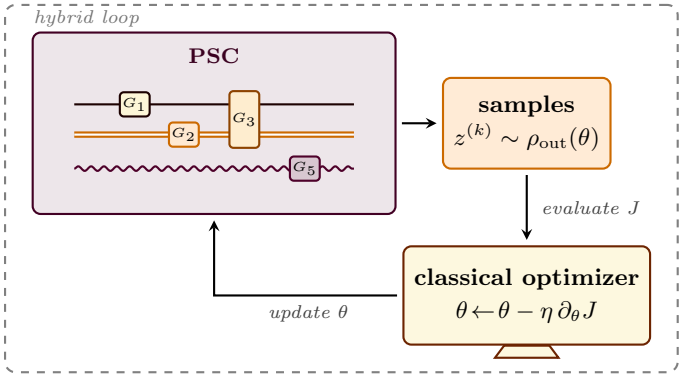


FIG. 5: The PSC (top-left) is sampled to produce samples $z^{(k)} \sim \rho_{\text{out}}(\theta)$ (top-right); a classical optimizer (bottom-right) evaluates the scalar loss $J(\theta)$ from those samples and updates θ for the next iteration.

The circuit is executed to produce samples $z^{(k)} \sim \rho_{\text{out}}(\theta)$, which are used to estimate the objective $J(\theta)$ of Eq. (32), and the gradient estimators of Section III C supply the parameters update for the kernels, with JAX providing the automatic differentiation and compilation machinery.

B. Backends

Once a logical PSC has been constructed, one has to choose a *backend* which specifies how the *sample* calls of a circuit are executed. `torx` provides three software backends. `SampleSimulator` draws a sample from ρ_{out} by sampling sequentially through the gates, and `HybridSampleSimulator` mirrors it for registers that mix discrete and continuous wires. `StateVectorSimulator` instead propagates the full probability vector, returning ρ_{out} exactly up to floating-point error. Since the vector has size 2^n for n pbits, it is primarily useful for small-scale prototyping.

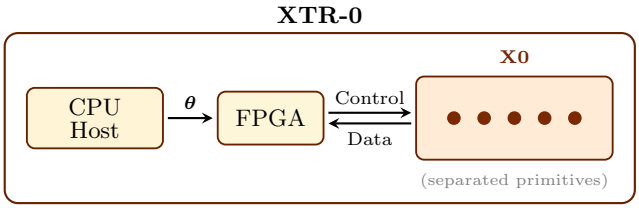


FIG. 6: The XTR-0 desktop platform. A CPU host and FPGA control physically separated pbit, pdit, Gaussian, and mixture-of-Gaussian primitives on the socketed X0 chip. The FPGA mediates data and control between primitives, so the logical interaction topology and schedule are software-defined. The outer box marks the complete XTR-0 platform.

A hardware backend in `torx` replaces the software pseudorandom generator (e.g. Threefry) with a physical randomness source. Any gate that needs a random draw can source it directly in hardware. The sigmoid-mixture gates of Section III A illustrate this concretely. A gate of the form of Eq. (17) is realized by programming a pbit (or pdit) to the mixture weight, then routing the register's values according to its outcome.

Concretely, we obtain randomness from X0, a subthreshold CMOS test chip accessed through the XTR-0 desktop platform, carrying pbits, pdits, Gaussian samplers, and mixture-of-Gaussian samplers as physically separated primitives. The host and FPGA set their parameters and route samples and control, so the logical interaction topology is software-defined (Figure 6). Operating transistors in the subthreshold regime exposes thermal and shot noise that X0 pbits use as a physical Bernoulli source, at energy costs of hundreds of attojoules to single femtojoules per sample at rates of tens of MHz [8, 9], comparable to superparamagnetic tunnel junctions [31–33], while leveraging CMOS only process steps. However, the output statistics of such devices drift from device to device. For this reason, before sampling for the first time each pbit undergoes a Bayesian-optimization-based calibration that maps digital controls to bias voltage, with typical per-site calibration errors around 5% in absolute probability. After calibration, a user programs target probabilities $p \in [0, 1]$ directly and receives binary outputs at rates set by the device relaxation time (∼100 ns). Appendix D details the X0 probabilistic primitives and the XTR-0 platform.

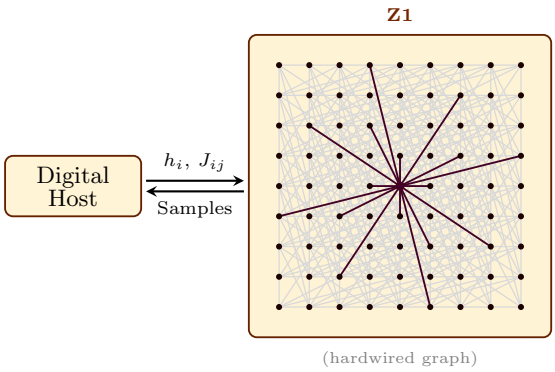


FIG. 7: Z1 architecture overview. The digital host communicates bias fields h_i and couplings J_{ij} directly to the Z1 chip for Gibbs sampling. The chip hardwires a degree-16 interaction graph in silicon, each pbit (black dot) is physically coupled to 16 neighbors via on-die wiring that extends beyond nearest-neighbor positions. Fuchsia edges highlight one node's full neighborhood.

Our next generation chip, which we refer to as Z1, hardwires a degree-16 sparse interaction graph directly in silicon and executes chromatic Gibbs sweeps without FPGA mediation, communicating only h_i and J_{ij} between sweeps (Figure 7, [9]). The two designs expose the substrate locality of Section III in opposite ways. XTR-0 reconstructs any k-local generator off-chip via the FPGA, paying a per-gate round-trip latency for arbi-

trary topology, while Z1 evaluates 2-local Glauber generators natively in a single sweep but requires compilation of the logical PSC to a hardware-compatible representation [20]. The same logical PSC compiles to different gate schedules on each chip, and `torx` keeps the support I_j of every gate visible so that this compilation is explicit. Section V F tests whether the X0 primitives provide Bernoulli randomness with statistics sufficient for the sampling algorithms of Section V.

V. DEMONSTRATIONS

We demonstrate the `torx` framework on a series of example sized to showcase the capabilities that a PSC can express rather than to show an efficiency or scalability advantage over standard implementations. Executable notebooks, including larger-scale variants of several of these examples, are available in the `torx` documentation [17], and we encourage readers to experiment with the framework directly.

The six demonstrations are: diffusion on a weighted graph (§V A); discrete diffusion for image generation (§V B); stochastic graph networks (§V C); a regime-switching jump-diffusion process (§V D); sampling and training an Ising model on a spin ring (§V E); and the same Ising sampling task driven by hardware randomness (§V F). All demonstrations use software pseudorandomness except the last, which sources its randomness from calibrated pbits on X0.

A. Random Walks on Graphs

Consider an undirected weighted graph $\mathcal{G} = (V, E)$ with edge weights $w_{ij} = w_{ji} \geq 0$. The graph Laplacian $L = D - A$ satisfies $L\mathbf{1} = 0$, so $Q = -L$ is a valid continuous-time Markov generator and the distribution $x(t)$ over vertices satisfies $\dot{x} = Qx$ with exact solution $x(t) = e^{Qt}x(0)$.

We can write the *edge decomposition*

$$Q = \sum_{\{i,j\}\in E} Q_{ij}, \qquad Q_{ij} = -w_{ij}(e_i - e_j)(e_i - e_j)^\top, \quad (45)$$

where each Q_{ij} is a local operator acting only on coordinates i and j. Since $\mathsf{PSWAP}_{ij}(p) = I - p(e_i - e_j)(e_i - e_j)^\top$, setting $p = w_{ij}\tau$ gives

$$\mathsf{PSWAP}_{ij}(w_{ij}\tau) = I + \tau Q_{ij}. \quad (46)$$

The exact exponential corresponds to $\mathsf{PSWAP}_{ij}\left(\frac{1-e^{-2w_{ij}\tau}}{2}\right) = e^{\tau Q_{ij}}$. Since Q_{ij} and Q_{kl} commute whenever edges $\{i,j\}$ and $\{k,l\}$ are vertex-disjoint (they act on disjoint coordinates), an edge coloring of $\mathcal{G}$ groups commuting generators together, reducing the number of Trotter groups to $k = \chi'(\mathcal{G})$ the edge chromatic number.

Applying Trotterization (Section III B) with Euler approximations $I + Q_{ij}t/N$, $k = \chi'(\mathcal{G})$ local generators and N steps gives a global error

$$\left\| e^{Qt} - \Big(\prod_{\{i,j\}\in E} (I + Q_{ij}t/N) \Big)^N \right\| = O\left(\frac{k^2t^2}{N}\right), \quad (47)$$

with each factor $I + Q_{ij}t/N$ implemented as a single PSWAP_{ij} gate.

While applying Trotterization with exact exponential $e^{t/NQ_{ij}}$, k local generators and N steps gives a global error

$$\left\| e^{Qt} - \Big(\prod_{\{i,j\}\in E} e^{Q_{ij}t/N} \Big)^N \right\| = O\left(\frac{k^2t^2}{N}\right), \quad (48)$$

with each factor $e^{Q_{ij}t/N}$ implemented as a single PSWAP_{ij} gate. Using Strang splitting gives an error of $O(k^3t^3/N^2)$ at the cost of $2k - 1$ gates per step rather than k.

As an example, consider the graph in Figure 8.

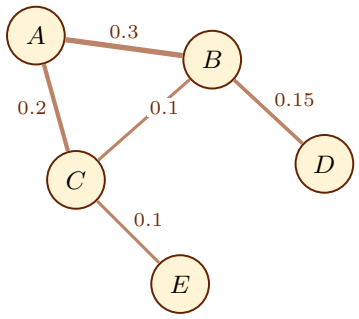


FIG. 8: Weighted graph used for the diffusion example.

a. Simulation. Taking initial conditions $x(0) = (1, 0, 0, 0, 0)$, corresponding to unit mass on state A, we obtain a circuit consisting of a single walk layer of the form shown in Figure 9. Applying Trotterization with Euler discretization and iterating up to $t = 10$, the resulting evolution of $x(t)$ is plotted in Figure 10.

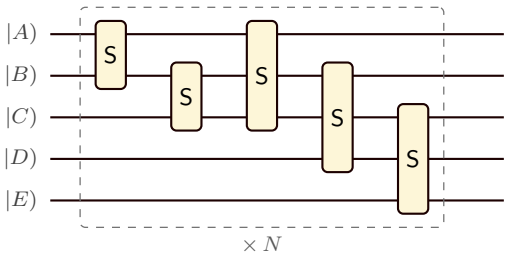


FIG. 9: Single step of Trotterized circuit for graph diffusion. Each box, labeled S, is a PSWAP gate acting on the pair of pbits it spans with probability proportional to the inverse number of Trotter steps; the dashed block is repeated N times.

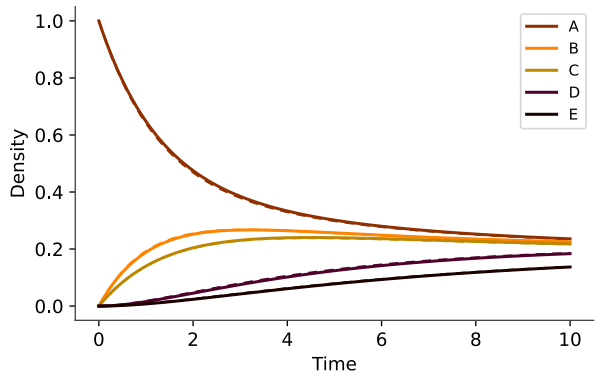


FIG. 10: Long term graph diffusion trajectory. Trotter with Euler discretization is shown in dashed markings. Exact solution is shown in solid lines of the same color.

B. Discrete Diffusion

Discrete diffusion models [34–37] corrupt data with a forward Markov Chain and learn a neural network to reverse it. Tau-leaping [38] is an approximate simulation method that advances a jump process by a fixed time interval, treating many small events within that interval as independent. We show that the tau-leaping reverse step can be implemented as a PSC of independent PNOT gates, and that adding a small number of PCNOT gates restores local pixel correlations that tau-leaping discards.

For the forward process, each bit $x_i \in \{0,1\}$ evolves independently under the uniform rate matrix $Q = \left(\begin{smallmatrix} -1 & 1 \\ 1 & -1 \end{smallmatrix}\right)$, giving transition kernel

$$e^{\sigma Q} = \begin{pmatrix} \frac{1}{2} + \frac{1}{2}e^{-2\sigma} & \frac{1}{2} - \frac{1}{2}e^{-2\sigma} \\ \frac{1}{2} - \frac{1}{2}e^{-2\sigma} & \frac{1}{2} + \frac{1}{2}e^{-2\sigma} \end{pmatrix}. \tag{49}$$

The off-diagonal entry $\frac{1}{2}(1-e^{-2\sigma})$ is the probability that a bit has flipped by noise level σ.

A convolutional denoiser [39] trained with binary cross-entropy produces per-pixel estimates $\hat{p}_i = P(x_i^{(0)} = 1 \mid x_t)$. The per-pixel flip probability for one reverse step is

$$p_{\text{flip}}^{(i)} = \begin{cases} 1 - \hat{p}_i & x_i = 1, \\ \hat{p}_i & x_i = 0, \end{cases} \tag{50}$$

and the corresponding PNOT logit is $\theta^{(i)} = \log\big(p_{\text{flip}}^{(i)}/(1 - p_{\text{flip}}^{(i)})\big)$. The tau-leaping reverse step is therefore exactly the PSC, $\bigotimes_i \mathsf{PNOT}(\theta^{(i)})$. It is composed of one independent gate per pixel, parametrized by the denoiser output.

Because tau-leaping treats all bits as independent, it misses joint structure. For each pixel pair (i,j) we compute the coupling

$$c_{j\to i} = p_{\text{flip}}^{(i)}(x_t \mid \bar{j}) - p_{\text{flip}}^{(i)}(x_t), \tag{51}$$

where $\bar{j}$ denotes x_t with bit j flipped, measuring how much j's state shifts i's flip probability. We select the top $K = 50$ pairs with the maximum $|c_{j\to i}|$ and append one $\mathsf{PCNOT}(c_{j\to i})$ per pair, giving a circuit of $n+K$ gates in total.

Results on 28×28 binary MNIST are shown in Figure 11: the PSC augmentation reduces bit-error rate from 0.114 (tau-leaping) to 0.113 and closes 41% of the FID gap between corrupted and clean distributions, demonstrating that denoiser outputs can parameterize structured PSC proposals.

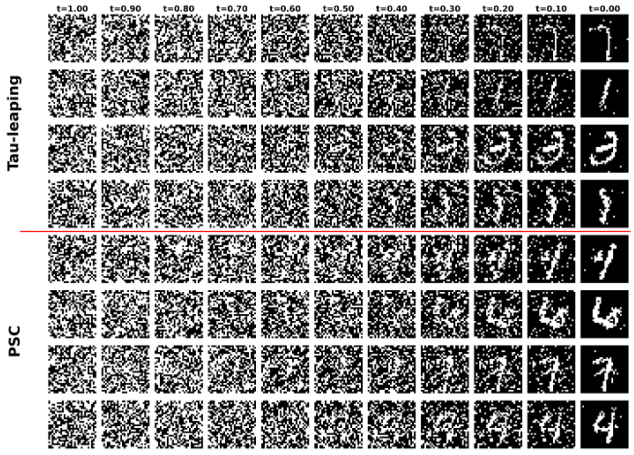


FIG. 11: A comparison of resulting images from the tau-leaping inference method and the augmented PSC method. We show the results for 10 solver steps.

C. Stochastic Graph Networks

Stochastic geometric deep learning [40, 41] builds symmetry into a stochastic model rather than learning it from data. The model is constrained to be equivariant under a group G of transformations of the input. In our implementation of a stochastic graph network as a PSC this is realized by parameter sharing where the same local gate kernel is reused wherever a group element relates two regions of the wire layout.

A *Stochastic Graph Network* (SGNN) instantiates this for the symmetry group of graph automorphisms [42, 43]: one wire per node, one parametrized PIsing gate per edge, with parameters tied under any automorphism that permutes the edges. Assigning pbit $\sigma_i \in \{0,1\}$ to node i with spin $s_i = 2\sigma_i - 1 \in \{-1,+1\}$, the gate for edge (i,j) is

$$G_{ij}(\theta_{ij}) = \mathsf{PIsing}(J_{ij},\, h_i,\, h_j,\, \beta,\, \Delta t), \tag{52}$$

where $J_{ij} \in \mathbb{R}$ is a learnable edge coupling and $(\beta, \Delta t)$ are fixed hyperparameters. One *sweep* is a series of gate

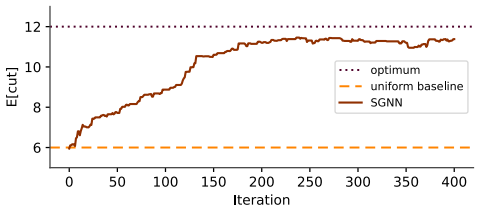


FIG. 12: Expected cut value during SGNN training on a random 3-regular 8-site graph ($|E| = 12$, optimal cut $= 12$). One PIsing gate per edge, $N = 5$ sweeps, $S = 1024$ samples per step. Dashed line: uniform random baseline; dotted line: optimum.

layers acting on a color partition k of the graph,

$$\widetilde{L}(\theta) = \Big(\bigotimes_k G_k(\theta)\Big) \otimes \mathbb{I}_{\setminus k}, \tag{53}$$

and the full SGNN circuit repeats the sweep N times.

Writing $\mathrm{cut}(\boldsymbol{\sigma}) = \sum_{(i,j)\in E}(1 - s_i s_j)/2$, the objective is to maximize the expected cut under the circuit output $\rho_\theta = \mathcal{G}_{\mathrm{SGNN}}(\theta)\,\rho_0$:

$$\mathcal{L}(\theta) = -\mathbb{E}_{\rho_\theta}[\mathrm{cut}] = -\sum_{(i,j)\in E} \frac{1 - \langle s_i s_j\rangle_{\rho_\theta}}{2}. \tag{54}$$

The circuit runs in sample mode with $\mathcal{L}$ estimated by Monte Carlo and edge couplings are optimized by a Metropolis search, which is one of several applicable optimization strategies; gradient-based alternatives such as REINFORCE or the parameter-shift rule of Section III C are equally applicable. At each step a Gaussian perturbation is proposed to θ, S samples are drawn from the updated circuit, the average cut is estimated, and the proposal is accepted with probability $\min(1, e^{-\Delta\mathcal{L}/T})$. On a random 3-regular graph ($n = 8$, $|E| = 12$) with one PIsing gate per edge, $N = 5$ sweeps, and $S = 1024$ samples per step, the trained SGNN outperforms a uniform sampler and recovers the optimal cut in the majority of runs (Figure 12). For context, the Goemans–Williamson SDP relaxation [44] guarantees a 0.878-approximation in the worst case over all graphs; this is a different kind of claim than the instance-specific result reported here, where the trained SGNN matches the optimal cut on this graph but carries no worst-case guarantee.

D. Jump Diffusion

Regime-switching models are a classical family of hybrid Markov processes in which a latent discrete state modulates the dynamics of a continuous variable [5]. The joint state $(S_t, X_t) \in \{0, \ldots, K-1\} \times \mathbb{R}$ evolves under

$$\mathrm{d}X_t = \mu_{S_t}\,\mathrm{d}t + \sigma_{S_t}\,\mathrm{d}W_t, \tag{55}$$

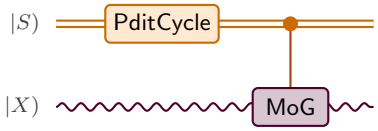


FIG. 13: One Trotter step for the circuit. The PditCycle gate transitions the pdit, then the MoG gate applies regime-conditioned Gaussian dynamics to the continuous wire. Repeating this unit $N = T/\Delta t$ times yields the full circuit.

where S_t is a continuous-time Markov chain, and the drift μ_k and volatility σ_k are conditioned on the discrete state, as in Eq. (27). This structure results in what is effectively a hidden Markov model (HMM) in continuous time, the observed process X_t is driven by a latent chain S_t.

The generator of the full continuous time process decomposes as $A = A_S + A_X$, where A_S governs discrete transitions and A_X generates the conditioned diffusion. These terms do not commute because the drift and volatility in A_X depend on the discrete state S. Writing $T_G(\Delta t)$ for the finite-Δt gate implementing generator G, we split the one-step propagator into two local gates:

$$T_{\mathrm{Trotter}}(\Delta t) = T_{\mathsf{MoG}}(\Delta t) T_{\mathsf{PditCycle}}(\Delta t). \tag{56}$$

The first factor discretizes the chain. We use a PditCycle gate on a K-state wire, which implements a cyclic random walk with forward rate λ_+ and backward rate λ_- which determine the probability of moving forward, cycling backward, or staying. in the current position $p_\pm = \lambda_\pm \Delta t$, $p_0 = 1 - (\lambda_+ + \lambda_-)\Delta t$. This results in an Euler discretization as we have seen before.

The second factor applies a MoG gate, the hybrid gate of Eq. (27), conditioned on the current $S = k$:

$$X' = X + \mu_k\,\Delta t + \sigma_k \sqrt{\Delta t}\mathcal{N}(0, 1). \tag{57}$$

This is an Euler step for the diffusion in Eq. (55). The full one-step circuit is shown in Figure 13.

To test this example, we set $K = 3$ regimes with drifts $\mu = (1.0, 0.0, -0.8)$, volatilities $\sigma = (0.3, 1.0, 0.5)$, and transition rates $\lambda_+ = 0.5$, $\lambda_- = 0.3$, simulating to time $T = 10$. Figure 14 shows sample trajectories colored by the latent regime at each step. Since A_S and A_X do not commute, the Trotterization incurs an error linear in Δt, which can be reduced by decreasing the step size or moving to a higher-order method.

E. Ising Sampling

The Ising model assigns probability $\pi(\boldsymbol{s}) \propto e^{-\beta H(\boldsymbol{s})}$ to binary configurations via the Hamiltonian $H(\boldsymbol{s}) = -\sum_{(i,j)\in E} J_{ij} s_i s_j - \sum_i h_i s_i$, $s_i \in \{-1, +1\}$. We demonstrate on an 8-site ring with random couplings J_{ij} and fields h_i at $\beta = 1.5$.

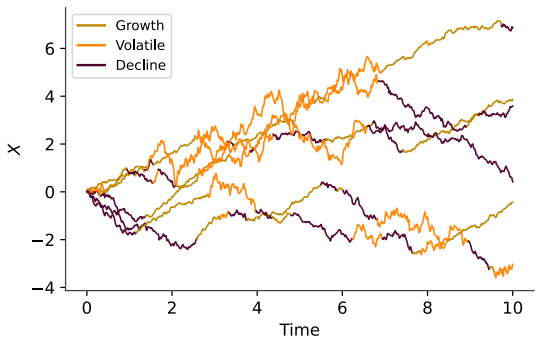


FIG. 14: Sample trajectories from the jump diffusion circuit, colored by the latent at each step.

Since the ring is two-colorable, even-indexed sites $\{0,2,4,6\}$ and odd-indexed sites $\{1,3,5,7\}$ each form a conditionally independent class [45]. Writing $\sigma_i = (s_i+1)/2 \in \{0,1\}$ for the circuit variable, the Gibbs conditional at site i is

$$\ell_i = h_i + \sum_{j\in N(i)} J_{ij}(2\sigma_j - 1), \tag{58}$$

$$\pi(\sigma_i = 1 \mid \sigma_{N(i)}) = \frac{1}{1+e^{-2\beta\ell_i}}. \tag{59}$$

One sweep applies two layers in sequence; within each layer every site in that color class is updated in parallel by

$$\mathsf{PColor}_i = \mathsf{PNOT}(2\beta\ell_i) \circ \mathsf{PReset}(\infty), \tag{60}$$

which resets σ_i to 0 and resamples it from the exact Gibbs conditional. For this we need to compute ℓ_i on the fly at each sweep, which we do classically in Python, although on our Z1 chip this can be done natively [20]. Over 6000 chains and 240 sweeps this achieves a total-variation distance of 0.052 from the exact Boltzmann distribution.

The parameters (J_{ij}, h_i) can be fit to data using Persistent Contrastive Divergence [46], the special case of the energy-based-kernel gradient of Section III C in which the kernel samples from the Ising Gibbs conditional: 2048 persistent chains run 2 chromatic sweeps per step to estimate model expectations, giving the log-likelihood gradient

$$\frac{\partial \log \mathcal{L}}{\partial J_{ij}} = \beta\big(\langle s_i s_j\rangle_{\text{data}} - \langle s_i s_j\rangle_{\text{model}}\big), \tag{61}$$

and analogously for h_i. Over 300 steps at learning rate 0.08 the learned parameters converge to the true couplings; see Figure 15.

A natural alternative assigns one PIsing gate per edge and Trotterizes their composition. As $N \to \infty$, this converges to the semigroup generated by $\sum_{(i,j)\in E} Q_{ij}$, but this generator is not the Glauber generator for the full ring. Each two-site generator Q_{ij} gives site i a flip rate that depends only on the single neighbor j via

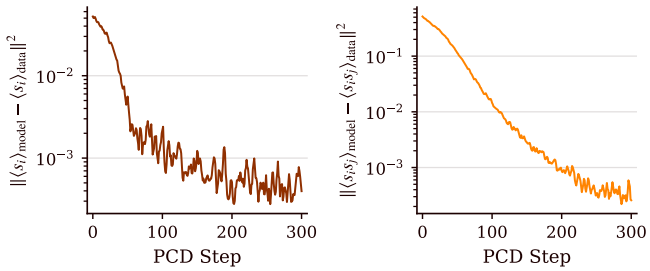


FIG. 15: MSE of learned parameters relative to the true Boltzmann distribution during PCD training on the 8-site ring. Left: magnetization h_i; right: pairwise couplings J_{ij}.

$\sigma\big(-2\beta J_{ij}(2\sigma_j - 1)\big)$; the effective single-site rate from composing all edge generators is $\sum_{j\in N(i)} \sigma\big(-2\beta J_{ij}(2\sigma_j - 1)\big)$. The correct Glauber rate is $\sigma(-2\beta\ell_i)$ with the full local field $\ell_i = h_i + \sum_{j\in N(i)} J_{ij}(2\sigma_j - 1)$ inside the sigmoid. Since σ is nonlinear, $\sum_j \sigma(\alpha_j) \neq \sigma(\sum_j \alpha_j)$ in general, so the stationary distribution of $\sum_{\text{edges}} Q_{ij}$ is not the Boltzmann distribution. On the same 8-site ring this yields TV distance 0.402, nearly eight times worse than chromatic Gibbs.

The exact Gibbs conditional can instead be embedded into a single gate acting on site i and all its d neighbors. This $2^{d+1}\times 2^{d+1}$ matrix stores $\mathsf{PNOT}(2\beta\ell_i)\circ\mathsf{PReset}(\infty)$ for each of the 2^d neighbor configurations statically, unlike PColor_i whose ℓ_i is recomputed dynamically at each sweep. Figure 16 confirms that the per-site gate matches the exact distribution, while edge-tiled PIsing does not. The same gate applies directly to any Ising-type inference task, including Boltzmann Machines [47] and DTMs [9].

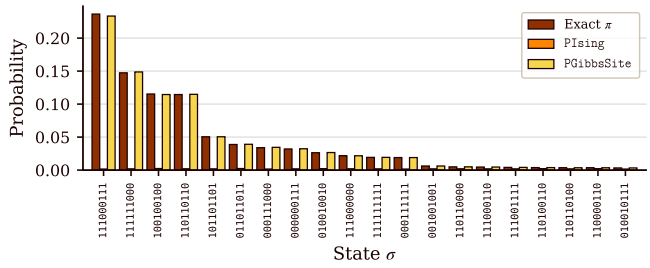
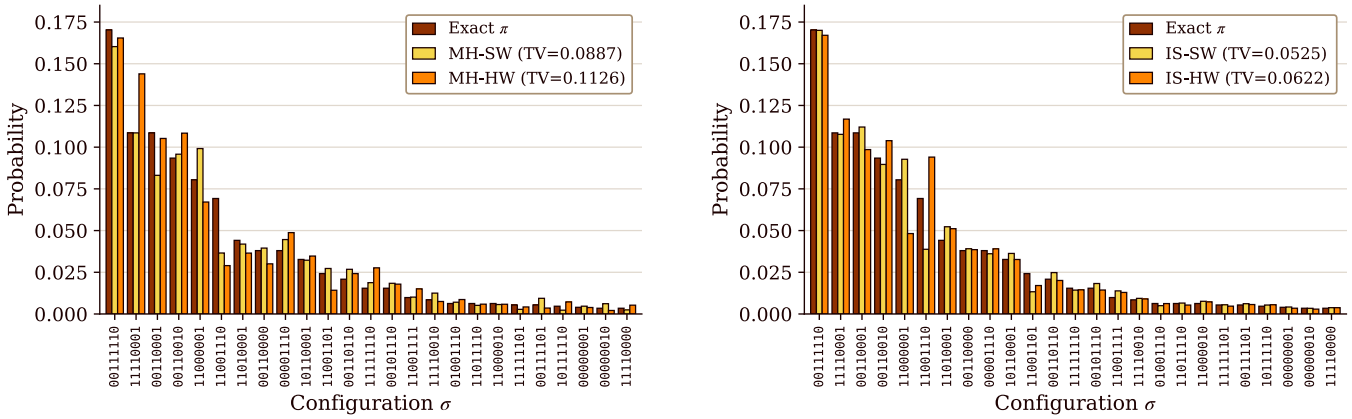


FIG. 16: State probabilities on the 8-site ring comparing the exact Boltzmann distribution, edge-tiled PIsing, and the per-site Gibbs gate. States are ordered by exact probability.

F. Ising Sampling with Hardware Randomness

We sample the stationary distribution π of the same 8-site Ising ring at $\beta = 1.5$ of Section V E using Bernoulli randomness from a calibrated pbit on X0, accessed through XTR-0, with proposals $q(\cdot;\theta)$



(a) Sampling with a Metropolis-Hastings method.

(b) Sampling with importance reweighting.

FIG. 17: Experimental validation of sampling with noise from XTR-0. The exact distribution is shown in dark red, with NumPy-based pseudo-randomness in yellow, and hardware randomness in orange (MH and IS). The target distribution is an 8-site Ising ring (with $\beta = 1.5$ and 20,000 samples) using a calibrated pbit on X0 within the XTR-0 platform.

drawn as independent per-site Bernoulli samples at the pbits' programmable biases θ_i. Two algorithms are tested: Metropolis-Hastings (MH) [48, 49], which uses hardware bits to inform the acceptance ratio $a = \min\left(1, \frac{\pi(x')}{\pi(x)} \frac{q(x)}{q(x')}\right)$, and self-normalized importance sampling (IS) [50, 51], which weights each sample by $\tilde{w}(m) = \pi(x^{(m)})/q(x^{(m)};\theta)$. The exact distribution π is available by full enumeration over the 2^8 configurations. Both methods achieve a total variation distance consistent with software baselines, using 20,000 hardware-drawn samples (Figure 17), demonstrating that analog hardware provides randomness of competitive quality to its digital counterpart.

VI. CONCLUSIONS

We have introduced Parametrized Stochastic Circuits as a gate-based model for programmable stochastic dynamics, together with `torx` [17], a differentiable JAX framework for building, executing, and training them. In this model, local stochastic kernels with tunable settings compose into a circuit operator, the gradient estimators of Section III C make the circuit trainable from samples, and a backend specifies the execution rule. The interface supports exact probability-vector simulation, sample-based execution, and hardware-sourced randomness while preserving the same logical PSC object.

The gate library covers directly specified stochastic kernels, generator-derived kernels such as PIsing, and Trotterized approximations of local continuous-time Markov generators. The demonstrations use these kernels for random walks on graphs, discrete diffusion, stochastic graph networks, jump diffusion and Ising sampling. In the final demonstration, calibrated subthreshold-CMOS pbits on X0 supply Bernoulli randomness to the same Ising sampling task, achieving total-variation distances consistent with software baselines. The logical circuits are unchanged, and the only difference between them is the randomness source.

The companion manuscript [20] on the `thermalizers` framework introduces a complementary hardware route, in which clamping the input variables of an energy-based model and sampling its output variables realizes a stochastic kernel directly as physical dynamics. In this relation, `torx` specifies and trains the logical PSC kernel, while the `thermalizers` construction implements it natively in hardware. Beyond this, several directions remain open: executing PSCs natively on Z1, a general theory of synthesizing a target kernel from a fixed gate set, and scaling the demonstrations beyond the toy sizes considered here. We view the PSC framework and `torx` as a concrete starting point for programming probabilistic hardware.

VII. ACKNOWLEDGMENTS

The authors thank Jack Ceroni and Edward Jiang for their contributions to early prototypes of the software, the authors also thank Nahuel Freitas and Andraž Jelinčič for their valuable feedback.

[1] J. Ho, A. Jain, and P. Abbeel, Denoising diffusion probabilistic models, in *Advances in Neural Information Processing Systems* (2020).

[2] R. M. Neal, MCMC using Hamiltonian dynamics, Handbook of Markov Chain Monte Carlo (2011).

[3] Y. Choi, A. Vergari, and G. Van den Broeck, Probabilistic circuits: A unifying framework for tractable probabilistic models, UCLA Computer Science Technical Report (2020).

[4] A. Vergari, Y. Choi, A. Liu, S. Teso, and G. Van den Broeck, A compositional atlas of tractable circuit operations: From simple transformations to complex information-theoretic queries, arXiv preprint arXiv:2102.06137 (2021).

[5] J. D. Hamilton, A new approach to the economic analysis of nonstationary time series and the business cycle, Econometrica: Journal of the Econometric Society , 357 (1989).

[6] S. Chowdhury, A. Grimaldi, N. A. Aadit, S. Niazi, M. Mohseni, S. Kanai, H. Ohno, S. Fukami, L. Theogarajan, G. Finocchio, *et al.*, A full-stack view of probabilistic computing with p-bits: Devices, architectures, and algorithms, IEEE Journal on Exploratory Solid-State Computational Devices and Circuits **9**, 1 (2023).

[7] B. Zhang, Y. Liu, T. Gao, J. Yin, Z. Guan, D. Zhang, and L. Zeng, Automatic extraction and compensation of p-bit device variations in large array utilizing boltzmann machine training, Micromachines **16**, 133 (2025).

[8] N. Freitas, G. Massarelli, J. Rothschild, D. Keane, E. Dawe, S. Hwang, A. Garlapati, and T. McCourt, Taming nonequilibrium thermal fluctuations in subthreshold cmos circuits, Phys. Rev. Appl. **25**, 034061 (2026).

[9] A. Jelinčič, O. Lockwood, A. Garlapati, P. Schillinger, I. Chuang, G. Verdon, and T. McCourt, An efficient probabilistic hardware architecture for diffusion-like models, arXiv preprint arXiv:2510.23972 (2025).

[10] O. Lockwood, J. Béjanin, J. Bus, C. Chamberland, P. Huembeli, F. Schäfer, and G. Verdon, A blueprint for equilibrium-based differentiable continuous-variable thermodynamic computing (2026), arXiv:2607.16183 [cs.LG].

[11] J. Schulman, N. Heess, T. Weber, and P. Abbeel, Gradient estimation using stochastic computation graphs, in *Advances in Neural Information Processing Systems*, Vol. 28 (Curran Associates, Inc., 2015).

[12] E. Bingham, J. P. Chen, M. Jankowiak, F. Obermeyer, N. Pradhan, T. Karaletsos, R. Singh, P. Szerlip, P. Horsfall, and N. D. Goodman, Pyro: Deep universal probabilistic programming, Journal of Machine Learning Research **20**, 1 (2019).

[13] D. Phan, N. Pradhan, and M. Jankowiak, Composable effects for flexible and accelerated probabilistic programming in NumPyro (2019), arXiv:1912.11554 [stat.ML].

[14] J. Lao, C. Suter, I. Langmore, C. Chimisov, A. Saxena, P. Sountsov, D. Moore, R. A. Saurous, M. D. Hoffman, and J. V. Dillon, tfp.mcmc: Modern markov chain monte carlo tools built for modern hardware (2020), arXiv:2002.01184 [stat.CO].

[15] M. Broughton, G. Verdon, T. McCourt, A. J. Martinez, J. H. Yoo, *et al.*, TensorFlow Quantum: A software framework for quantum machine learning, arXiv preprint arXiv:2003.02989 (2020).

[16] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, S. Ahmed, V. Ajith, M. S. Alam, G. Alonso-Linaje, B. Akash-Narayanan, A. Asadi, *et al.*, PennyLane: Automatic differentiation of hybrid quantum-classical computations, arXiv preprint arXiv:1811.04968 (2018).

[17] Extropic AI, Torx, documentation and runnable examples at `https://torx-docs.vercel.app/examples.html`.

[18] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, JAX: composable transformations of Python+NumPy programs (2018).

[19] P. Kidger and C. Garcia, Equinox: neural networks in JAX via callable PyTrees and filtered transformations, arXiv preprint arXiv:2111.00254 (2021).

[20] M. Amico, A. Jelinčič, C. O. Nancarrow, L. Tyrpak, D. Roberts, S. Morton, D. Sakthivadivel, A. Gopal, and G. Verdon, Thermalizing stochastic programs (2026), arXiv:2608.01615 [cs.ET].

[21] C. Duffee, J. Athas, A. Grimaldi, D. Volpe, G. Finocchio, E. Wei, and P. Khalili Amiri, Extended-variable probabilistic computing with p-dits, arXiv preprint arXiv:2506.00269 (2025).

[22] J. R. Norris, *Markov Chains* (Cambridge University Press, Cambridge, UK, 1997).

[23] C. M. Bishop, *Pattern Recognition and Machine Learning*, Information Science and Statistics (Springer, New York, 2006).

[24] J. E. Johnson, Markov-type Lie groups in GL (n, R), Journal of Mathematical Physics **26**, 252 (1985).

[25] D. A. Levin, M. J. Luczak, and Y. Peres, Glauber dynamics for the mean-field Ising model: cut-off, critical power law, and metastability (2007), arXiv:0712.0790 [math.PR].

[26] H. F. Trotter, On the product of semi-groups of operators, Proceedings of the American Mathematical Society **10**, 545 (1959).

[27] G. Strang, On the construction and comparison of difference schemes, SIAM Journal on Numerical Analysis **5**, 506 (1968).

[28] M. Suzuki, Fractal decomposition of exponential operators with applications to many-body theories and Monte Carlo simulations, Physics Letters A **146**, 319 (1990).

[29] R. J. Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning, Machine Learning **8**, 229 (1992).

[30] E. Greensmith, P. L. Bartlett, and J. Baxter, Variance reduction techniques for gradient estimates in reinforcement learning, Journal of Machine Learning Research **5**, 1471 (2004).

[31] B. Parks, M. Bapna, J. Igbokwe, H. Almasi, W. Wang, and S. A. Majetich, Superparamagnetic perpendicular magnetic tunnel junctions for true random number generators, AIP Advances **8** (2018).

[32] D. Vodenicarevic, N. Locatelli, A. Mizrahi, T. Hirtzlin, J. S. Friedman, J. Grollier, and D. Querlioz, Circuit-level evaluation of the generation of truly random bits with superparamagnetic tunnel junctions, in *2018 IEEE International Symposium on Circuits and Systems (ISCAS)* (IEEE, 2018) pp. 1–4.

[33] M. W. Daniels, A. Madhavan, P. Talatchian, A. Mizrahi, and M. D. Stiles, Energy-efficient stochastic computing with superparamagnetic tunnel junctions, Physical Review Applied **13**, 034016 (2020).

[34] J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and R. van den Berg, Structured denoising diffusion models in discrete state-spaces, in *Advances in Neural Information Processing Systems*, Vol. 34 (2021).

[35] X. Wang, Z. Zheng, F. Ye, D. Xue, S. Huang, and Q. Gu, Diffusion language models are versatile protein learners, arXiv preprint arXiv:2402.18567 (2024).

[36] S. S. Sahoo, M. Arriola, Y. Schiff, A. Gokaslan, E. Marroquin, J. T. Chiu, A. Rush, and V. Kuleshov, Simple and effective masked diffusion language models, Advances in Neural Information Processing Systems **37**, 130136 (2024).

[37] S. Nie, F. Zhu, Z. You, X. Zhang, J. Ou, J. Hu, J. Zhou, Y. Lin, J.-R. Wen, and C. Li, Large language diffusion models, arXiv preprint arXiv:2502.09992 (2025).

[38] D. T. Gillespie, Approximate accelerated stochastic simulation of chemically reacting systems, The Journal of Chemical Physics **115**, 1716 (2001).

[39] O. Ronneberger, P. Fischer, and T. Brox, U-Net: Convolutional networks for biomedical image segmentation, in *International Conference on Medical image computing and computer-assisted intervention* (Springer, 2015) pp. 234–241.

[40] M. M. Bronstein, J. Bruna, T. Cohen, and P. Veličković, Geometric deep learning: Grids, groups, graphs, geodesics, and gauges, arXiv preprint arXiv:2104.13478 (2021).

[41] T. S. Cohen and M. Welling, Group equivariant convolutional networks, in *Proceedings of the 33rd International Conference on Machine Learning (ICML)* (2016) pp. 2990–2999.

[42] G. Verdon, T. McCourt, E. Luzhnica, V. Singh, S. Leichenauer, and J. Hidary, Quantum graph neural networks, arXiv preprint arXiv:1909.12264 (2019).

[43] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun, Graph neural networks: A review of methods and applications, AI open **1**, 57 (2020).

[44] M. X. Goemans and D. P. Williamson, Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming, Journal of the ACM **42**, 1115 (1995).

[45] J. Gonzalez, Y. Low, A. Gretton, and C. Guestrin, Parallel Gibbs sampling: From colored fields to thin junction trees, in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics* (PMLR, 2011) pp. 324–332.

[46] T. Tieleman, Training restricted Boltzmann machines using approximations to the likelihood gradient, in *Proceedings of the 25th International Conference on Machine Learning* (Helsinki, Finland, 2008).

[47] D. H. Ackley, G. E. Hinton, and T. J. Sejnowski, A learning algorithm for Boltzmann machines, Cognitive science **9**, 147 (1985).

[48] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, Equation of state calculations by fast computing machines, The Journal of Chemical Physics **21**, 1087 (1953).

[49] W. K. Hastings, Monte Carlo sampling methods using Markov chains and their applications, Biometrika **57**, 97 (1970).

[50] H. Kahn and T. E. Harris, Estimation of particle transmission by random sampling, National Bureau of Standards Applied Mathematics Series **12**, 27 (1951).

[51] T. Kloek and H. K. Van Dijk, Bayesian estimates of equation system parameters: an application of integration by Monte Carlo, Econometrica: Journal of the Econometric Society , 1 (1978).

[52] M. Benedetti, E. Lloyd, S. Sack, and M. Fiorentini, Parameterized quantum circuits as machine learning models, Quantum science and technology **4**, 043001 (2019).

[53] G. Verdon, M. Broughton, and J. Biamonte, A quantum algorithm to train neural networks using low-depth circuits, arXiv preprint arXiv:1712.05304 (2017).

[54] G. Verdon, J. Pye, and M. Broughton, A universal training algorithm for quantum deep learning, arXiv preprint arXiv:1806.09729 (2018).

[55] G. Verdon, M. Broughton, J. R. McClean, K. J. Sung, R. Babbush, Z. Jiang, H. Neven, and M. Mohseni, Learning to learn with quantum neural networks via classical neural networks, arXiv preprint arXiv:1907.05415 (2019).

[56] G. Verdon, J. Marks, S. Nanda, S. Leichenauer, and J. Hidary, Quantum Hamiltonian-based models and the variational quantum thermalizer algorithm, arXiv preprint arXiv:1910.02071 (2019).

[57] Extropic AI, Inside x0 and xtr-0, https://extropic.ai/writing/inside-x0-and-xtr-0 (2025), documentation; accessed July 2026.

Appendix A: Additional Gates

Here we list some additional elementary gates that one might consider (and are supported by `torx`). We note that all of the following gates we implement can be written as,

$$\mathsf{ApplyB}(p) = (1-p)\mathbb{I} + pB,$$

where ApplyB is the name of the operation and it applies B with probability p, otherwise does nothing. Setting $p \in \{0, 1\}$ in the following gates leads to deterministic gate.

a. PReset Sets the pbit to 0 with probability p, otherwise does nothing.

$$\mathsf{PReset}(p) = \begin{pmatrix} 1 & p \\ 0 & 1-p \end{pmatrix}$$

b. PCNOT Flips the second pbit if the first is 1, with probability p.

$$\mathsf{PCNOT}(p) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1-p & p \\ 0 & 0 & p & 1-p \end{pmatrix}$$

c. PSWAP Swaps two pbits with probability p, otherwise does nothing.

$$\mathsf{PSWAP}(p) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1-p & p & 0 \\ 0 & p & 1-p & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

d. PJUMP Moves probability from $|10\rangle$ to $|01\rangle$ with probability p.

$$\mathsf{PJUMP}(p) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & p & 0 \\ 0 & 0 & 1-p & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

e. PDEMUX Copies first pbit to second pbit and resets input to 0 with probability p, otherwise does nothing (a probabilistic demultiplexor).

$$\mathsf{PDEMUX}(p) = \begin{pmatrix} 1 & p & 0 & 0 \\ 0 & 1-p & p & p \\ 0 & 0 & 1-p & 0 \\ 0 & 0 & 0 & 1-p \end{pmatrix}$$

f. PCopy Copies first pbit to second pbit with probability p, otherwise does nothing.

$$\mathsf{PCopy}(p) = \begin{pmatrix} 1 & p & 0 & 0 \\ 0 & 1-p & 0 & 0 \\ 0 & 0 & 1-p & 0 \\ 0 & 0 & p & 1 \end{pmatrix}$$

g. PditShift Shifts a k-dimensional pdit cyclically $i \to (i-1) \bmod k$ with probability p. For $k = 3$:

$$\mathsf{PditShift}(p) = \begin{pmatrix} 1-p & p & 0 \\ 0 & 1-p & p \\ p & 0 & 1-p \end{pmatrix}$$

h. PditSWAP Swaps two k-dimensional pdits with probability p. The $k^2 \times k^2$ matrix permutes $(i, j) \leftrightarrow (j, i)$:

$$\mathsf{PditSWAP}(p) = (1-p)\mathbb{I} + pS$$

where S is the swap permutation matrix.

Appendix B: Guide for readers from Quantum Information

The visual similarity of PSCs to Parametrized Quantum Circuits (PQCs) [52], composition of basic gates to create circuits, hides several substantive distinctions in data encoding, circuit-run accounting, and gradient estimation. The variational loop of Figure 5 is deliberately the stochastic analog of the parametrized-quantum-circuit training loops developed in the quantum machine learning literature [15, 53, 54], where a parametrized unitary is sampled, a classical optimizer evaluates a scalar loss, and the circuit parameters are updated. Indeed, several of the constructions we use have direct precursors in that body of work: variational training of a parametrized circuit [53, 54], classical meta-optimizers that warm-start the circuit parameters [55], symmetry-tied gates on a graph [42, 43], and the preparation of thermal (Gibbs) states by a variational procedure [56]. Software frameworks such as TensorFlow-Quantum [15] and PennyLane [16] were built to express exactly such hybrid quantum-classical loops and to differentiate through them, and the PSC software model can be read as a stochastic-matrix counterpart to them. The remainder of this section makes the distinctions precise.

In quantum information, computations are encoded as unitary matrices, whereas in Parametrized Stochastic Circuits computations are encoded as stochastic matrices. These two classes of objects differ in fundamental ways, which constrain the sorts of computations that quantum and stochastic computers can perform. First, a stochastic matrix need not have an inverse, unlike a unitary. The deterministic reset of one bit to 0 is

$$K_{\text{reset}} = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}, \qquad K_{\text{reset}}|0\rangle = K_{\text{reset}}|1\rangle = |0\rangle; \quad \text{(B1)}$$

which has rank 1 and therefore no inverse. Mixing, marginalization, and coarse-graining are likewise valid stochastic circuit operations that generally cannot be inverted. Second, the two models differ in how information is read out. The state of a quantum register is a vector of complex amplitudes, so a readout requires the choice of a measurement basis, and the measurement collapses the state, such that subsequent operations act on

the post-measurement state rather than on the original one. In contrast, the state of a PSC register is a classical probability distribution, and a readout is simply a sample drawn from it. Drawing a sample does not alter the distribution, so one can take measurements at any point in the circuit without affecting the remainder of the computation.

Appendix C: Pmode Gate Library and Constructions

This appendix collects the detailed gate library for (pmode) introduced in Section III A.

1. Gaussian Stochastic Maps

A pmode transformation $\mathcal{T} : \mathcal{P}(\mathbb{R}^N) \to \mathcal{P}(\mathbb{R}^N)$ is *Gaussian* if it sends every Gaussian density to a Gaussian density. Equivalently, it is *affine in moments*: there exist $M \in \mathbb{R}^{N\times N}$, $\boldsymbol{d} \in \mathbb{R}^N$, $\Delta \in \mathrm{Sym}_N^{\geq 0}(\mathbb{R})$ with

$$\boldsymbol{\mu} \longmapsto \boldsymbol{\mu}' = M\boldsymbol{\mu} + \boldsymbol{d}, \qquad \Sigma \longmapsto \Sigma' = M\Sigma M^\top + \Delta. \tag{C1}$$

Collecting the data of a Gaussian stochastic map as a triple $(M, \boldsymbol{d}, \Delta)$ and equipping it with the composition law

$$(M_2, \boldsymbol{d}_2, \Delta_2) \circ (M_1, \boldsymbol{d}_1, \Delta_1) = \big(M_2 M_1,\ M_2\boldsymbol{d}_1 + \boldsymbol{d}_2,\ M_2\Delta_1 M_2^\top + \Delta_2\big) \tag{C2}$$

gives a semigroup of Gaussian stochastic maps, denoted here by $\mathcal{S}_N$. Composition is associative, the identity is $(\mathbb{I}, \mathbf{0}, 0)$, and $(M, \boldsymbol{d}, \Delta)$ has a two-sided inverse only when $\Delta = 0$ and M is invertible; any nonzero noise injection is irreversible at the level of Markov kernels. Classical stochastic systems live on configuration space alone, not on phase space, so the symplectic form does not appear and three independent ingredients, linear map A, drift b, and noise injection D, replace the single Hamiltonian generator of the CV-quantum case.

The finite-time propagator of a linear SDE $\mathrm{d}X_t = (AX_t + b)\,\mathrm{d}t + B\,\mathrm{d}W_t$ is the one-parameter sub-semigroup of $\mathcal{S}_N$ given by

$$\begin{aligned} M(t) &= e^{At}, \\ \boldsymbol{d}(t) &= \int_0^t e^{A(t-s)} b\,\mathrm{d}s, \\ \Delta(t) &= \int_0^t e^{As}\, D\, e^{A^\top s}\,\mathrm{d}s, \end{aligned} \tag{C3}$$

with $D = BB^\top$. The moments of a Gaussian initial condition obey $\dot{\boldsymbol{\mu}}_t = A\boldsymbol{\mu}_t + b$ and the differential Lyapunov equation $\dot{\Sigma}_t = A\Sigma_t + \Sigma_t A^\top + D$; Gaussian pmode calculations therefore reduce to finite-dimensional linear algebra.

a. SVD-based Gaussian-channel factorization. Any $(M, \boldsymbol{d}, \Delta) \in \mathcal{S}_N$ with invertible M factors as

$$\begin{aligned} (M, \boldsymbol{d}, \Delta) = (\mathbb{I}, \boldsymbol{d}_2, \Delta_2) \circ (U_2, \mathbf{0}, 0) \\ \circ (\Sigma_S, \mathbf{0}, \Delta_S) \circ (V^\top, \mathbf{0}, 0) \circ (\mathbb{I}, \boldsymbol{d}_1, 0), \end{aligned} \tag{C4}$$

where $M = U_2\Sigma_S V^\top$ is the SVD with $U_2, V \in \mathrm{O}(N)$, $\Sigma_S = \mathrm{diag}(\sigma_1, \ldots, \sigma_N)$, Δ_S is diagonal in the singular-vector basis, and Δ_2 absorbs residual cross-mode noise when such a decomposition of the noise covariance is available. Each orthogonal factor can be decomposed into planar two-mode rotations. This gives a factorization of linear Gaussian stochastic maps in terms of rotations, scalings, drifts, and noise injections. Unlike Bloch–Messiah decompositions in continuous-variable quantum optics, this factorization does not use a symplectic structure.

2. Gaussian Gate Library

a. $\mathsf{PDisp}(\alpha)$ Shifts the mean by α: $\mathcal{N}(\mu, \sigma^2) \mapsto \mathcal{N}(\mu + \alpha, \sigma^2)$. Triple $(1, \alpha, 0)$.

b. $\mathsf{PScale}(r)$ Scales mean and variance: $\mathcal{N}(\mu, \sigma^2) \mapsto \mathcal{N}(e^r\mu, e^{2r}\sigma^2)$. Triple $(e^r, 0, 0)$. Reversible; no noise injection.

c. $\mathsf{PMix}_{ij}(\theta)$ Planar rotation on two modes: $(X_i, X_j) \mapsto (\cos\theta\, X_i - \sin\theta\, X_j, \sin\theta\, X_i + \cos\theta\, X_j)$. Triple $(R(\theta), \mathbf{0}, 0)$ with $R(\theta) \in \mathrm{SO}(2)$.

d. $\mathsf{P2Sq}_{ij}(r)$ Hyperbolic rotation on two modes: $(X_i, X_j) \mapsto (\cosh r\, X_i + \sinh r\, X_j, \sinh r\, X_i + \cosh r\, X_j)$. Induces $\mathrm{Cov}(X_i', X_j') = \sinh(2r)$ for independent $\mathcal{N}(0, 1)$ inputs.

Together with interferometer decompositions, these gates realize every element of $\mathcal{S}_N$ with $\Delta = 0$ at depth $O(N)$.

3. Generator-derived gates: Fokker–Planck building blocks

a. Backward vs forward generators. The Fokker–Planck (forward) generator $\mathcal{L}^\dagger$ acts on densities; the backward (Kolmogorov) generator $\mathcal{L}$ is its formal adjoint and acts on observables:

$$\mathcal{L}^\dagger p = -\nabla\cdot\big[\boldsymbol{\mu}(x)\, p\big] + \tfrac{1}{2}\sum_{ij}\partial_i\partial_j\big[D_{ij}(x)\, p\big], \tag{C5}$$

$$\mathcal{L} f = \boldsymbol{\mu}(x)\cdot\nabla f + \tfrac{1}{2}\sum_{ij} D_{ij}(x)\,\partial_i\partial_j f, \tag{C6}$$

related by $\langle f, \mathcal{L}^\dagger p\rangle = \langle \mathcal{L} f, p\rangle$. The finite-time propagator is $P_t = \exp(t\mathcal{L}^\dagger)$ on densities and $\exp(t\mathcal{L})$ on observables; we report $\mathcal{L}$ for each gate below. For linear drift and constant diffusion, $\mathcal{L}^\dagger$ preserves the Gaussian manifold and the $\mathcal{S}_N$-triple of Eq. (C3) is the propagator data.

b. Drift gate $\mathsf{PDrift}(v,t)$. $\mathcal{L} = v\,\partial_x$; SDE $\mathrm{d}X_t = v\,\mathrm{d}t$; propagator $p(x) \mapsto p(x - vt)$; triple $(1, vt, 0)$. A generator-time-parametrized form of PDisp suited to Trotter schedules.

c. Diffusion gate $\mathsf{PDiff}(D,t)$. $\mathcal{L} = D\,\partial_x^2$; SDE $\mathrm{d}X_t = \sqrt{2D}\,\mathrm{d}W_t$; Gaussian convolution kernel $K_t(y \mid x) = \mathcal{N}(y; x, 2Dt)$; triple $(1, 0, 2Dt)$. No stationary distribution on $\mathbb{R}$; the canonical noise-injection generator gate.

d. Ornstein–Uhlenbeck gate $\mathsf{POU}(\gamma, D, t)$. $\mathcal{L} = -\gamma\, x\,\partial_x + D\,\partial_x^2$; SDE $\mathrm{d}X_t = -\gamma X_t\,\mathrm{d}t + \sqrt{2D}\,\mathrm{d}W_t$; kernel

$$K_t(y \mid x_0) = \mathcal{N}\Big(y;\ x_0 e^{-\gamma t},\ \tfrac{D}{\gamma}\big(1 - e^{-2\gamma t}\big)\Big); \tag{C7}$$

triple $\big(e^{-\gamma t}, 0, \frac{D}{\gamma}(1 - e^{-2\gamma t})\big)$; stationary distribution $\mathcal{N}(0, D/\gamma)$. A non-trivial element of $\mathcal{S}_1$ with both deterministic contraction ($M = e^{-\gamma t} < 1$) and noise injection ($\Delta > 0$); the pmode generator-derived example analogous to PIsing for pbit dynamics.

e. Geometric Brownian motion gate $\mathsf{PGBM}(\mu, \sigma, t)$. $\mathcal{L} = \mu x\,\partial_x + \frac{\sigma^2 x^2}{2}\partial_x^2$ on $x > 0$; SDE $\mathrm{d}X_t = \mu X_t\,\mathrm{d}t + \sigma X_t\,\mathrm{d}W_t$ (multiplicative noise); closed form $X_t = X_0 \exp((\mu - \sigma^2/2)t + \sigma W_t)$. Generally not in $\mathcal{S}_1$ due to state-dependent diffusion.

f. Cox–Ingersoll–Ross gate $\mathsf{PCIR}(\kappa, \theta, \sigma, t)$. $\mathcal{L} = \kappa(\theta - x)\,\partial_x + \frac{\sigma^2 x}{2}\partial_x^2$ on $x \geq 0$ with Feller condition $2\kappa\theta \geq \sigma^2$; SDE $\mathrm{d}X_t = \kappa(\theta - X_t)\,\mathrm{d}t + \sigma\sqrt{X_t}\,\mathrm{d}W_t$; stationary distribution in the Gamma or non-central-χ^2 family.

g. Exchange generator. $\mathcal{L}_{\mathrm{exch}} = \gamma(x_j - x_i)\partial_{x_i} + \gamma(x_i - x_j)\partial_{x_j} + D(\partial_{x_i}^2 + \partial_{x_j}^2)$ drives two coupled OU processes conserving $x_i + x_j$.

h. Correlated-noise generator. $\mathcal{L}_{\mathrm{corr}} = D\partial_{x_i}^2 + D\partial_{x_j}^2 + 2\rho D\partial_{x_i}\partial_{x_j}$ is pure noise injection with covariance $D\left(\begin{smallmatrix} 1 & \rho \\ \rho & 1 \end{smallmatrix}\right)$ per unit time.

i. Disjoint-support tensor identity. If $\mathcal{L}_A, \mathcal{L}_B$ are Fokker–Planck operators with disjoint mode supports, then

$$\exp\big[(\mathcal{L}_A \otimes \mathbb{I}_B + \mathbb{I}_A \otimes \mathcal{L}_B)\,t\big] = \exp(\mathcal{L}_A\,t) \otimes \exp(\mathcal{L}_B\,t), \tag{C8}$$

the continuous-variable analog of the disjoint-support identity used throughout Sections III–III B and the basis for parallel scheduling of pmode gates.

j. Hamiltonian–Langevin and BAOAB decomposition. Underdamped Langevin dynamics in $(\boldsymbol{q}, \boldsymbol{p})$-space with potential $U(\boldsymbol{q})$,

$$\begin{aligned} \mathrm{d}\boldsymbol{q} &= (\boldsymbol{p}/m)\,\mathrm{d}t, \\ \mathrm{d}\boldsymbol{p} &= -\nabla U(\boldsymbol{q})\,\mathrm{d}t - \gamma\boldsymbol{p}\,\mathrm{d}t + \sqrt{2\gamma m k_B T}\,\mathrm{d}\boldsymbol{W}, \end{aligned} \tag{C9}$$

has Fokker–Planck generator $\mathcal{L} = \mathcal{L}_{\mathrm{drift}} + \mathcal{L}_{\mathrm{fric}} + \mathcal{L}_{\mathrm{diff}}$ with

$$\mathcal{L}_{\mathrm{drift}} = -(\boldsymbol{p}/m)\cdot\nabla_{\boldsymbol{q}} + \nabla U(\boldsymbol{q})\cdot\nabla_{\boldsymbol{p}}, \tag{C10}$$

$$\mathcal{L}_{\mathrm{fric}} = \gamma\,\nabla_{\boldsymbol{p}}\cdot(\boldsymbol{p}\,\cdot), \tag{C11}$$

$$\mathcal{L}_{\mathrm{diff}} = \gamma m k_B T\,\nabla_{\boldsymbol{p}}^2. \tag{C12}$$

Each generator maps to a gate: $\mathcal{L}_{\mathrm{fric}}$ gives a PScale (momentum contraction), $\mathcal{L}_{\mathrm{diff}}$ gives a PDiff with $D = \gamma m k_B T$, and $\mathcal{L}_{\mathrm{drift}}$ gives a symplectic step. The BAOAB Strang integrator applies these in order: momentum half-step (B), position half-step (A), OU step (O), position half-step (A), momentum half-step (B). The sampler compiles to an alternating sequence of PDrift, PScale, and PDiff gates, with Tier-3 $\mathsf{CFunc}[-\nabla U]$ gates when U is non-quadratic.

4. Tier hierarchy of non-Gaussian primitives

We organize pmode constructions into tiers by the resources they consume: Tier 1 comprises the noiseless Gaussian gates of C 2; Tier 2 adds prepared-and-discarded ancillas, continuous (2a) or discrete (2b); Tier 3 adds measurement and classical feedback.

a. Tier 2a: noise via auxiliary pmodes. For input $X \sim \mathcal{N}(\mu, \sigma^2)$ and independent ancilla $A \sim \mathcal{N}(0, \sigma_A^2)$, applying $\mathsf{PMix}(\theta)$ and discarding A gives

$$X' \sim \mathcal{N}\big(\cos\theta\cdot\mu,\ \cos^2\theta\,\sigma^2 + \sin^2\theta\,\sigma_A^2\big). \tag{C13}$$

Correlated multi-mode noise is realized by correlating auxiliaries with $\mathsf{P2Sq}(r)$ before mode-by-mode mixing; the resulting covariance $\sin^2\theta\,\sigma_A^2\left(\begin{smallmatrix} \cosh 2r & \sinh 2r \\ \sinh 2r & \cosh 2r \end{smallmatrix}\right)$ realizes any 2×2 noise injection by tuning (θ, r, σ_A).

b. Tier 2b: Gaussian mixtures via auxiliary pdits. The controlled-shift gate is

$$\begin{aligned} \mathsf{CShift}(\boldsymbol{\alpha}):\ |k) \otimes p(x) &\longmapsto |k) \otimes p(x - \alpha_k), \\ \boldsymbol{\alpha} &= (\alpha_0, \ldots, \alpha_{d-1}). \end{aligned} \tag{C14}$$

Preparing a pdit with PMF $\pi = (\pi_0, \ldots, \pi_{d-1})$, applying $\mathsf{CShift}(\boldsymbol{\alpha})$ to (K, X) with $X \sim \mathcal{N}(\mu, \sigma^2)$, and marginalizing K yields

$$p_{X'}(x) \;=\; \sum_{k=0}^{d-1} \pi_k\,\mathcal{N}(x; \mu + \alpha_k, \sigma^2), \tag{C15}$$

a Gaussian mixture with shared variance. The companion controlled-scale gate $\mathsf{CScale}(\boldsymbol{r})$: $|k) \otimes p(x) \mapsto |k) \otimes e^{-r_k} p(e^{-r_k}x)$ gives heteroscedastic mixtures $\sum_k \pi_k \mathcal{N}(e^{r_k}\mu, e^{2r_k}\sigma^2)$. Finite Gaussian mixtures can approximate broad classes of continuous densities, including compactly supported densities in L^1 under standard regularity assumptions. Tier-2b therefore gives a practical density-approximation route by adding one type of discrete ancilla and one controlled gate.

c. Tier 3: measurement-feedback for arbitrary nonlinearities. For state-dependent nonlinearities the measurement gate $\mathsf{Meas} : \mathsf{pmode} \to \mathsf{real}$ samples $x_0 \sim p(x)$ and produces a classical real; $\mathsf{CFunc}[f] : \mathsf{real} \to \mathsf{real}$ evaluates an arbitrary measurable f on a classical coprocessor; and $\mathsf{FeedbackDisp} : (\mathsf{real}, \mathsf{pmode}) \to \mathsf{pmode}$ treats the classical value as a deterministic displacement on the live pmode. The composition Meas →

CFunc[ϕ] $\rightarrow$ Inject realizes the pushforward $\phi_* p_X$; inserting a PDiff step recovers an overdamped Langevin update $X_{t+\Delta t} = X_t - \Delta t\, \nabla V(X_t) + \sqrt{2\Delta t}\, \xi_t$ that samples from the Gibbs distribution $\pi(x) \propto e^{-V(x)}$. Tier 3 incurs the round-trip latency cost of each measurement and classical evaluation; this compilation strategy is reserved for state-dependent nonlinearities that cannot be approximated by a finite mixture within the application accuracy budget.

5. Hybrid pdit/pmode gates and jump-diffusion

The general jump-diffusion process

$$\begin{aligned} \mathrm{d}X_t &= \mu_{N_t}(X_t)\,\mathrm{d}t + \sigma_{N_t}(X_t)\,\mathrm{d}W_t, \\ N_t &: i \to j \text{ at rate } \lambda_{ji}(X_t), \end{aligned} \tag{C16}$$

has joint generator

$$\mathcal{L}_{\mathrm{JD}} = \sum_i |i\rangle\langle i| \otimes \mathcal{L}_i^{(\mathrm{diff})} + \sum_{i \neq j} \lambda_{ji}(x) \left[|j\rangle\langle i| \otimes \mathcal{K}_{ji} - |i\rangle\langle i| \otimes \mathbb{I} \right], \tag{C17}$$

with diffusion piece $\mathcal{L}_i^{(\mathrm{diff})} = \mu_i(x)\partial_x + \frac{1}{2}\sigma_i^2(x)\partial_x^2$. Assuming constant rates and no reset, the Strang split

$$P_{\Delta t} \;\approx\; e^{\mathcal{L}^{(\mathrm{diff})}\Delta t/2}\, e^{\mathcal{L}^{(\mathrm{jump})}\Delta t}\, e^{\mathcal{L}^{(\mathrm{diff})}\Delta t/2} \tag{C18}$$

has local error $O(\Delta t^3)$ and global error $O(\Delta t^2)$; the corresponding circuit alternates CtrlOU (half-step), a pdit-only jump layer (full step), and CtrlOU (half-step). State-dependent rates $\lambda_{ji}(x)$ require a Tier-3 measurement of the pmode before each jump layer, and this hybrid Tier-2/Tier-3 schedule is a compilation for regime-switching dynamics with state-dependent rates that appear in financial, biophysical, and chemical models.

Appendix D: X0 sampling circuits and the XTR-0 platform

This appendix summarizes the X0 test chip and the XTR-0 platform used to host it in the hardware demonstration of Section V F, complementing the backend description of Section IV B. The device physics of the X0 primitives is developed in Ref. [8], and the platform is described in the documentation materials [57]. Here we collect the facts relevant to their use as a randomness source for PSCs.

a. Platform. XTR-0 is a desktop development platform consisting of a digital host, an FPGA, and two daughterboard sockets, each of which can receive a chip carrying probabilistic primitives. The socketed construction allows the same platform to host future chips without modification. The test chip currently hosted on the platform, the X0 chip, was fabricated in an advanced FinFET CMOS process [8] with the goal of validating transistor-level noise models and probabilistic circuit designs. Control parameters are set through on-chip 8-bit digital-to-analog converters, while the output signals, which are weak because the circuits operate in the subthreshold regime, are routed off-chip through a dedicated amplification chain and digitized on the daughterboard before reaching the host [8]. In this way, the host observes the analog dynamics of every primitive directly and can update parameters in a closed loop.

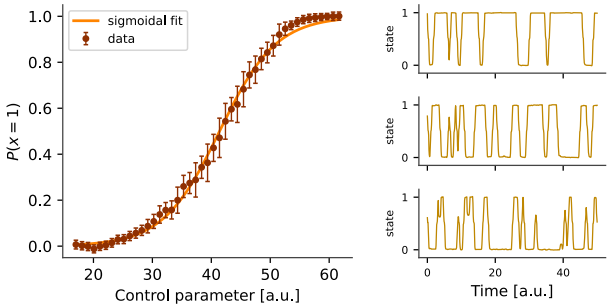


FIG. 18: Measured operating characteristic of an X0 pbit. The probability of observing state 1 is approximately sigmoidal in the control parameter (left); output waveforms at three bias settings show the corresponding telegraph signals (right). Data digitized from the published figure in the public X0/XTR-0 description.

b. Sampling model. Each primitive outputs a continuous-time random voltage signal whose stationary statistics realize a target distribution, such that observing the signal at intervals long compared to the relaxation time of the circuit yields approximately independent draws. For the pbit, the memory between observations separated by a lag τ is quantified by the normalized autocorrelation

$$r_{xx}(\tau) \approx e^{-\tau/\tau_0}, \tag{D1}$$

with $\tau_0 \approx 100$ ns for the device used here [9], while pbit designs on the X0 process span relaxation times from roughly 1 ns to 1 ms. For a representative Gaussian primitive, Ref. [8] reports correlation times down to $\tau \approx 1$ ns and an energy per independent sample, defined as the product of the DC power and the correlation time, of $E \approx 15$ aJ. These figures anchor the energy and rate ranges quoted in Section IV B. It is important to note that extreme values of the controls recover deterministic behavior: for example, the pbit becomes a constant 0 or 1 source.

c. Probabilistic primitives. The X0 primitives supply hardware draws for the wire types of Table I, and are summarized in Table II. The pbit outputs a two-level random telegraph signal whose bias is approximately sigmoidal in a single control voltage, as shown by the measurements of Figure 18. The pdit generalizes this behavior to a k-level signal, where normalization leaves

TABLE II: X0 sampling circuits exposed by XTR-0.

Circuit	Output signal	Distribution	Controls
pbit	2-level telegraph	Bernoulli	1 (bias)
pdit	k-level telegraph	categorical	$k-1$ (weights)
pmode	continuous	Gaussian	σ (1D); ρ (2D)
pMoG	multi-level	Gaussian mixture	π_j, μ_j, Σ_j (bounded)

$k-1$ independent controls. Furthermore, XTR-0 exposes a one-dimensional pmode with programmable standard deviation and a two-dimensional pmode with programmable correlation between its two output voltages. The pmode electronic circuit is a network of capacitively coupled CMOS inverters, referred to in Ref. [8] as a non-equilibrium adjustable-temperature resistor network (NEAT-RN) and identified there as the first electronic implementation of a Brownian gyrator using CMOS circuits. The pmode wire of Section II A is the software abstraction of this device. Finally, the mixture-of-Gaussians circuit (pMoG) jumps between levels whose positions, spreads, and weights are programmable within bounds.